

# Graphs — Complete Guide

---

A comprehensive placement + CP reference for graph fundamentals, representations, and traversal algorithms.

---

## 1. Directed Graph (Digraph)

---

### 1. Overview

A **directed graph** (or digraph) is a graph where edges have a direction — they go from one vertex to another. If you travel from A to B, you cannot necessarily travel from B to A unless there is a separate edge in the opposite direction.

Formally, a directed graph  $G = (V, E)$  where  $E$  is a set of ordered pairs  $(u, v)$ .

### 2. Intuition

Think of a **one-way street** in a city. You can drive from point A to point B, but you cannot drive back the same way unless there is a separate one-way street pointing the other way.

Another analogy: **Twitter follow relationships**. If you follow someone, they don't automatically follow you back. The relationship is directed.

#### Step-by-step reasoning:

- Every edge has a source and a destination.
- Traversal follows the arrow direction.
- The graph can have paths that exist only in one direction.
- Cycles exist only if you can return to a node by following edges in their correct direction.

### 3. When to Use It

Use a directed graph when relationships have a direction:

- **Dependency graphs** (task A must finish before task B starts)
- **Call graphs** (function A calls function B)
- **Social networks** (follow/follower relationships)
- **Web page links** (page A links to page B)
- **State machines** (transition from state S1 to S2)
- **Prerequisite chains** (course A is required for course B)

**Trigger phrases:** "dependency", "prerequisite", "follows", "calls", "points to", "flows from X to Y", "directed", "one-way", "topological order".

### 4. When Not to Use It

- When relationships are **inherently bidirectional** (friendship, road between two cities) — use an undirected graph instead.
- When direction doesn't matter and complicates the solution.
- If you need to traverse in both directions equally, an undirected graph halves the edges you need to store.

5. Core Concepts

| Concept        | Explanation                                              | Why It Matters                                               |
|----------------|----------------------------------------------------------|--------------------------------------------------------------|
| Out-degree     | Number of edges leaving a vertex                         | Tells how many outgoing connections a node has               |
| In-degree      | Number of edges entering a vertex                        | Tells how many incoming connections a node has               |
| Source         | Vertex with in-degree = 0                                | Starting point in topological sort                           |
| Sink           | Vertex with out-degree = 0                               | End point in dependency chains                               |
| Directed cycle | Path that returns to a node following edge directions    | Prevents topological ordering; indicates circular dependency |
| DAG            | Directed Acyclic Graph — a directed graph with no cycles | Foundation of topological sorting, DP on graphs              |

6. Step-by-Step Algorithm (Representation & Basic Operations)

Building a directed graph:

1. Choose a representation: adjacency list or adjacency matrix.
2. For each edge (u, v):
  - **Adjacency list:** Add v to adj[u].
  - **Adjacency matrix:** Set mat[u][v] = 1 (or weight).
3. Do NOT add u to adj[v] — that's the whole point of a directed graph.

Checking if an edge exists:

- Adjacency list: search adj[u] for v — O(V) worst case, O(1) average with hash set.
- Adjacency matrix: check mat[u][v] — O(1).

7. Dry Run

**Input:** Vertices = {0, 1, 2, 3}, Edges = [(0→1), (0→2), (1→2), (2→3), (3→1)]

| Step | Operation | adj[0] | adj[1] | adj[2] | adj[3] |
|------|-----------|--------|--------|--------|--------|
| 1    | Add 0→1   | [1]    | []     | []     | []     |
| 2    | Add 0→2   | [1, 2] | []     | []     | []     |
| 3    | Add 1→2   | [1, 2] | [2]    | []     | []     |

| Step | Operation | adj[0] | adj[1] | adj[2] | adj[3] |
|------|-----------|--------|--------|--------|--------|
| 4    | Add 2→3   | [1, 2] | [2]    | [3]    | []     |
| 5    | Add 3→1   | [1, 2] | [2]    | [3]    | [1]    |

**Edge exists check:** Does 1→0 exist? `adj[1] = [2]`, no 0 → no edge. Does 0→1 exist? `adj[0] = [1, 2]`, yes → edge exists.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DirectedGraph {
private:
    int V; // number of vertices
    vector<vector<int>> adj; // adjacency list

public:
    DirectedGraph(int vertices) : V(vertices) {
        adj.resize(V);
    }

    // Add a directed edge u -> v
    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    // Get neighbours of a vertex
    vector<int> getNeighbours(int u) {
        return adj[u];
    }

    // Check if edge u -> v exists
    bool hasEdge(int u, int v) {
        for (int x : adj[u])
            if (x == v) return true;
        return false;
    }

    // Get out-degree of a vertex
    int outDegree(int u) {
        return adj[u].size();
    }

    // Get in-degree of a vertex
    int inDegree(int u) {
        int deg = 0;
        for (int i = 0; i < V; i++)
            for (int v : adj[i])
                if (v == u) deg++;
    }
}
```

```

        return deg;
    }

    // Print graph
    void print() {
        for (int u = 0; u < V; u++) {
            cout << u << " -> ";
            for (int v : adj[u])
                cout << v << " ";
            cout << "\n";
        }
    }
};

int main() {
    DirectedGraph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 3);
    g.addEdge(3, 1);
    g.print();
    // Output:
    // 0 -> 1 2
    // 1 -> 2
    // 2 -> 3
    // 3 -> 1
    return 0;
}

```

## 9. Python Implementation

```

class DirectedGraph:
    def __init__(self, vertices):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_edge(self, u, v):
        """Add directed edge u -> v"""
        self.adj[u].append(v)

    def get_neighbours(self, u):
        return self.adj[u]

    def has_edge(self, u, v):
        return v in self.adj[u]

    def out_degree(self, u):
        return len(self.adj[u])

    def in_degree(self, u):

```

```

    deg = 0
    for i in range(self.V):
        for v in self.adj[i]:
            if v == u:
                deg += 1
    return deg

def print_graph(self):
    for u in range(self.V):
        print(f"{u} -> {' '.join(map(str, self.adj[u]))}")

# Example usage
g = DirectedGraph(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)
g.add_edge(3, 1)
g.print_graph()
# Output:
# 0 -> 1 2
# 1 -> 2
# 2 -> 3
# 3 -> 1

```

## 10. Code Explanation

- **adj vector of vectors:** Each index `u` stores a list of vertices `v` such that edge `u→v` exists.
- **addEdge(u, v):** Only adds `v` to `adj[u]`. No reverse edge.
- **hasEdge(u, v):** Linear scan of `adj[u]`. For faster lookup, use `unordered_set<int>` for each adjacency list.
- **inDegree(u):** Scans all adjacency lists counting occurrences of `u`. This is  $O(V+E)$  per call — if you need it often, maintain an `in_degree` array updated during `addEdge`.
- **outDegree(u):** Just returns `adj[u].size()` —  $O(1)$ .

## 11. Complexity Analysis

| Operation          | Adjacency List | Adjacency Matrix |
|--------------------|----------------|------------------|
| Add edge           | $O(1)$         | $O(1)$           |
| Remove edge        | $O(V)$         | $O(1)$           |
| Check edge exists  | $O(V)$         | $O(1)$           |
| Get all neighbours | $O(V)$         | $O(V)$           |
| Space              | $O(V + E)$     | $O(V^2)$         |

## 12. Common Patterns

| Pattern                              | Identification                                                  | Approach                                                 | Problems                                   |
|--------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------|
| <b>Topological Sort</b>              | Dependency ordering, prerequisite chains                        | Kahn's algorithm (BFS) or DFS post-order                 | Course Schedule II, Alien Dictionary       |
| <b>Cycle Detection</b>               | Circular dependencies, deadlock detection                       | DFS with recursion stack, Kahn's (count nodes processed) | Course Schedule, Find Eventual Safe States |
| <b>Longest Path in DAG</b>           | "Maximum time to complete all tasks", critical path             | Topological sort + DP relaxation                         | Parallel Courses, Minimum Time to Complete |
| <b>Strongly Connected Components</b> | Mutual reachability, "groups where everyone can reach everyone" | Kosaraju's / Tarjan's algorithm                          | Number of Provinces (directed version)     |

## 13. Common Mistakes

- Adding edge in both directions when implementing a directed graph.
- Forgetting that BFS/DFS on a directed graph only reaches nodes reachable via directed paths.
- Assuming `hasEdge(u, v)` implies `hasEdge(v, u)`.
- Ignoring self-loops (edges from a node to itself) unless the problem explicitly allows them.
- Not handling disconnected components when traversing.

## 14. Edge Cases

- **Empty graph** ( $V = 0$ )
- **Single vertex, no edges**
- **Graph with only self-loops**
- **Disconnected directed graph**
- **Graph with cycles**
- **DAG** (no cycles)
- **Complete directed graph** (every pair has edges in both directions)
- **Graph with parallel edges** (multiple edges from  $u$  to  $v$ )

## 15. Variations

| Variation                       | What Changes                      | When Used                                      | Importance |
|---------------------------------|-----------------------------------|------------------------------------------------|------------|
| <b>Transpose graph</b>          | Reverse all edges                 | Kosaraju's SCC algorithm, reverse reachability | High       |
| <b>Bipartite directed graph</b> | Edge direction respects partition | Special dependency problems                    | Medium     |
| <b>Multigraph</b>               | Multiple parallel edges allowed   | Network flow (multiple pipes)                  | Low        |
| <b>Hypergraph</b>               | Edges connect $>2$ vertices       | Specialized problems                           | Rare       |

## 16. Related Concepts

- **Undirected Graph** — Directionless; every edge is bidirectional.
- **DAG** — Directed graph with no cycles; enables topological ordering.
- **Topological Sort** — Linear ordering of DAG vertices respecting edge directions.
- **Strongly Connected Components** — Partitions of a directed graph where every node is mutually reachable within the component.

## 17. Practice Problems

### Easy

- **Find if Path Exists in Graph** — LeetCode (1971) — Basic directed traversal
- **Course Schedule** — LeetCode (207) — Cycle detection in directed graph

### Medium

- **Course Schedule II** — LeetCode (210) — Topological sort
- **Alien Dictionary** — LeetCode (269) / GFG — Topological sort on character ordering
- **All Paths From Source to Target** — LeetCode (797) — DFS on DAG

### Hard

- **Minimum Height Trees** — LeetCode (310) — Topological removal of leaves
- **Longest Increasing Path in a Matrix** — LeetCode (329) — DFS + memoization on DAG
- **Parallel Courses III** — LeetCode (2050) — Topological sort with DP

## 18. Interview Explanation

"A directed graph is a graph where edges have a direction. I represent it using an adjacency list, where each vertex stores a list of vertices it points to. This is useful for problems involving dependencies, prerequisites, or one-way relationships. Key operations include adding edges, checking if an edge exists, and computing in-degree/out-degree. For dependency problems, I use topological sorting via Kahn's algorithm or DFS with a visited stack."

## 19. Revision Notes

- Edge  $(u, v)$  means  $u \rightarrow v$ , NOT  $v \rightarrow u$ .
- Adjacency list: `adj[u].push_back(v)` only.
- In-degree = number of incoming edges; out-degree = number of outgoing edges.
- DAG = Directed Acyclic Graph = no cycles = can be topologically sorted.
- Cycle detection: track `recStack[]` in DFS or use Kahn's (count processed nodes).
- `adj[u].size()` = out-degree. In-degree requires scanning or a separate array.

## 20. Final Cheat Sheet

| Aspect      | Summary                                            |
|-------------|----------------------------------------------------|
| When to use | One-way relationships, dependencies, prerequisites |

| Aspect                | Summary                                                                              |
|-----------------------|--------------------------------------------------------------------------------------|
| <b>Representation</b> | Adjacency list: <code>vector&lt;vector&lt;int&gt;&gt; adj</code>                     |
| <b>Add edge</b>       | <code>adj[u].push_back(v)</code>                                                     |
| <b>Check edge</b>     | <code>find(adj[u].begin(), adj[u].end(), v)</code> or use <code>unordered_set</code> |
| <b>Complexity</b>     | $O(V+E)$ space, $O(1)$ add, $O(V)$ lookup (adj list)                                 |
| <b>Key traps</b>      | Don't add reverse edge, check for cycles, handle disconnected components             |
| <b>Must know</b>      | Topological sort, cycle detection, in-degree/out-degree                              |

## 2. Undirected Graph

### 1. Overview

An **undirected graph** is a graph where edges have no direction. If there is an edge between A and B, you can travel from A to B and from B to A using the same edge.

Formally,  $G = (V, E)$  where  $E$  is a set of **unordered pairs**  $\{u, v\}$ .

### 2. Intuition

Think of a **two-way road** connecting two cities. Traffic flows both ways.

Another analogy: **Facebook friendship**. If you are friends with someone, they are friends with you. The relationship is mutual.

#### Step-by-step reasoning:

- An edge  $\{u, v\}$  means  $u$  and  $v$  are connected.
- Traversal can go either way.
- If there's a path from A to B, there's also a path from B to A (the reverse path).
- A cycle exists if there are two different paths between any two vertices.

### 3. When to Use It

- **Friendship networks** (mutual relationships)
- **Road networks between cities** (bidirectional)
- **Maze or grid problems** (movement in 4 directions)
- **Minimum spanning tree** problems (Kruskal's, Prim's)
- **Connected components** detection
- **Bipartite graph** checking

**Trigger phrases:** "bidirectional", "mutual", "connected", "link", "friendship", "road between", "two-way", "undirected", "components", "spanning tree".

### 4. When Not to Use It

- When relationships are **naturally one-way** (dependencies, follow relationships) — use a directed graph.
- When direction matters for the problem (e.g., "prerequisite").
- If you use an undirected graph for a directed problem, you'll get wrong answers (e.g., treating "A requires B" as "A and B are connected").

5. Core Concepts

| Concept             | Explanation                                                          | Why It Matters                                                       |
|---------------------|----------------------------------------------------------------------|----------------------------------------------------------------------|
| Degree              | Number of edges incident to a vertex                                 | In undirected graphs, degree = in-degree = out-degree (no direction) |
| Connected component | A maximal set of vertices where every pair has a path between them   | Key for graph partitioning                                           |
| Bridge              | An edge whose removal disconnects the graph                          | Critical connections in networks                                     |
| Articulation point  | A vertex whose removal disconnects the graph                         | Single point of failure                                              |
| Tree                | Connected acyclic undirected graph                                   | Foundation for MST, DFS tree                                         |
| Bipartite graph     | Vertices can be partitioned into two sets with no edges inside a set | 2-coloring problems                                                  |

6. Step-by-Step Algorithm (Building & Basic Operations)

1. Choose representation: adjacency list or adjacency matrix.
2. For each edge (u, v):
  - **Adjacency list:** Add v to adj[u] and u to adj[v].
  - **Adjacency matrix:** Set mat[u][v] = mat[v][u] = 1.
3. Always add the reverse edge — this is the defining feature of undirected graphs.

7. Dry Run

**Input:** Vertices = {0, 1, 2, 3}, Edges = [(0,1), (0,2), (1,2), (2,3)]

| Step | Operation | adj[0] | adj[1] | adj[2]    | adj[3] |
|------|-----------|--------|--------|-----------|--------|
| 1    | Add (0,1) | [1]    | [0]    | []        | []     |
| 2    | Add (0,2) | [1, 2] | [0]    | [0]       | []     |
| 3    | Add (1,2) | [1, 2] | [0, 2] | [0, 1]    | []     |
| 4    | Add (2,3) | [1, 2] | [0, 2] | [0, 1, 3] | [2]    |

**Edge exists check:** Does 0→1 exist? Yes — adj[0] has 1 AND adj[1] has 0.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class UndirectedGraph {
private:
    int V;
    vector<vector<int>> adj;

public:
    UndirectedGraph(int vertices) : V(vertices) {
        adj.resize(V);

        // Add undirected edge u - v
        void addEdge(int u, int v) {
            adj[u].push_back(v);
            adj[v].push_back(u);    // <-- reverse edge added
        }

        vector<int> getNeighbours(int u) { return adj[u]; }

        bool hasEdge(int u, int v) {
            for (int x : adj[u])
                if (x == v) return true;
            return false;
        }

        int degree(int u) { return adj[u].size(); }

        void print() {
            for (int u = 0; u < V; u++) {
                cout << u << " : ";
                for (int v : adj[u])
                    cout << v << " ";
                cout << "\n";
            }
        }
    };

    int main() {
        UndirectedGraph g(4);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 2);
        g.addEdge(2, 3);
        g.print();
        // Output:
        // 0 : 1 2
        // 1 : 0 2
        // 2 : 0 1 3
        // 3 : 2
    }
}
```

```

    return 0;
}

```

## 9. Python Implementation

```

class UndirectedGraph:
    def __init__(self, vertices):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_edge(self, u, v):
        """Add undirected edge u - v"""
        self.adj[u].append(v)
        self.adj[v].append(u) # reverse edge

    def get_neighbours(self, u):
        return self.adj[u]

    def has_edge(self, u, v):
        return v in self.adj[u]

    def degree(self, u):
        return len(self.adj[u])

    def print_graph(self):
        for u in range(self.V):
            print(f"{u} : {' '.join(map(str, self.adj[u]))}")

# Example
g = UndirectedGraph(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)
g.print_graph()
# 0 : 1 2
# 1 : 0 2
# 2 : 0 1 3
# 3 : 2

```

## 10. Code Explanation

- **addEdge(u, v):** Adds **v** to **adj[u]** AND **u** to **adj[v]**. This ensures the edge is bidirectional.
- **degree(u):** Simply returns **adj[u].size()**. Since every edge is stored twice, the count is correct.
- **hasEdge(u, v):** Checks **adj[u]** for **v**. Since the graph is undirected, checking **adj[v]** for **u** gives the same result.
- **Edge duplication:** Every edge is stored twice — once in each direction. This doubles the space but makes traversal natural.

## 11. Complexity Analysis

| Operation      | Complexity                              |
|----------------|-----------------------------------------|
| Add edge       | $O(1)$                                  |
| Check edge     | $O(V)$ (adj list) / $O(1)$ (adj matrix) |
| Get neighbours | $O(\text{degree}(u))$                   |
| Space          | $O(V + 2E)$                             |

Note: Space is  $O(V + 2E)$  because each undirected edge is stored twice.

## 12. Common Patterns

| Pattern                     | Identification                    | Approach                                 | Problems                                  |
|-----------------------------|-----------------------------------|------------------------------------------|-------------------------------------------|
| <b>Connected Components</b> | Count groups, provinces           | DFS/BFS from each unvisited node         | Number of Provinces, Connected Components |
| <b>Bipartite Check</b>      | 2-coloring, team division         | BFS/DFS coloring with alternating colors | Is Graph Bipartite?                       |
| <b>Cycle Detection</b>      | Detect if graph has a cycle       | DFS with parent tracking                 | Redundant Connection, Cycle in Graph      |
| <b>Bridges</b>              | Critical connections              | Tarjan's algorithm with discovery time   | Critical Connections in a Network         |
| <b>MST</b>                  | Minimum cost to connect all nodes | Kruskal's (DSU) or Prim's (heap)         | Min Cost to Connect Points                |

## 13. Common Mistakes

- Forgetting to add the reverse edge — this makes the graph directed instead of undirected.
- Adding the reverse edge twice (duplicating edges) — leads to incorrect degree counts.
- Using `hasEdge(u, v)` and assuming it's  $O(1)$  when using a list (it's  $O(V)$ ).
- Not tracking parent in DFS cycle detection — a back edge to the parent is NOT a cycle in an undirected graph.
- Assuming the graph is always connected — always handle disconnected components.

## 14. Edge Cases

- **Single vertex, no edges** — degree = 0, 1 component.
- **Two vertices, one edge** — simplest connected graph.
- **Complete graph** — every pair connected.
- **Disconnected graph** — multiple components.
- **Graph with self-loops** ( $u, u$ ) — allowed in some problems.
- **Tree** — connected acyclic graph.
- **Graph with parallel edges** — allowed in multigraphs.

## 15. Variations

| Variation                        | What Changes                       | When Used                     | Importance |
|----------------------------------|------------------------------------|-------------------------------|------------|
| <b>Weighted undirected graph</b> | Each edge has a weight             | MST, shortest path            | High       |
| <b>Multigraph</b>                | Multiple parallel edges allowed    | Network flow                  | Low        |
| <b>Bipartite graph</b>           | Special 2-partition structure      | Matching problems             | High       |
| <b>Tree</b>                      | Connected acyclic undirected graph | Subproblem in many algorithms | Very High  |

## 16. Related Concepts

- **Directed Graph** — Edges have direction; undirected is a special case where edges go both ways.
- **DSU (Disjoint Set Union)** — Efficiently tracks connected components in undirected graphs.
- **BFS/DFS** — Graph traversal algorithms; work identically on undirected graphs.
- **Minimum Spanning Tree** — Only defined for undirected graphs.

## 17. Practice Problems

### Easy

- **Find the Town Judge** — LeetCode (997) — Degree counting
- **Number of Provinces** — LeetCode (547) — Connected components

### Medium

- **Redundant Connection** — LeetCode (684) — Cycle detection in undirected graph
- **Is Graph Bipartite?** — LeetCode (785) — 2-coloring
- **Number of Connected Components** — LeetCode (323) — DFS/BFS

### Hard

- **Critical Connections in a Network** — LeetCode (1192) — Bridges/Tarjan's algorithm
- **Min Cost to Connect All Points** — LeetCode (1584) — MST (Prim's/Kruskal's)
- **Most Stones Removed** — LeetCode (947) — DSU on grid/undirected graph

## 18. Interview Explanation

"An undirected graph is a graph where edges are bidirectional. I use an adjacency list representation where each edge is stored in both directions. Key applications include finding connected components, detecting cycles, checking bipartiteness, and computing minimum spanning trees. For cycle detection, I use DFS with a parent parameter to avoid false positives from the reverse edge."

## 19. Revision Notes

- Always add both directions: `adj[u].push_back(v)` and `adj[v].push_back(u)`.

- Degree = `adj[u].size()`.
- Cycle detection (DFS): skip the parent node when checking visited neighbours.
- Connected components = number of DFS/BFS runs needed to visit all vertices.
- Bipartite check: assign alternating colours (0/1) during BFS/DFS.
- Tree =  $V-1$  edges, connected, acyclic.

## 20. Final Cheat Sheet

| Aspect         | Summary                                                        |
|----------------|----------------------------------------------------------------|
| When to use    | Bidirectional/mutual relationships                             |
| Representation | <code>adj[u].push_back(v) + adj[v].push_back(u)</code>         |
| Add edge       | $O(1)$ — add both directions                                   |
| Check edge     | $O(V)$ in list, $O(1)$ in matrix                               |
| Space          | $O(V + 2E)$                                                    |
| Key traps      | Don't forget reverse edge; track parent in DFS cycle detection |
| Must know      | Connected components, bipartite check, cycle detection, MST    |

## 3. Weighted Graph

### 1. Overview

A **weighted graph** is a graph where each edge has a numerical value (weight) associated with it. Weights typically represent cost, distance, time, capacity, or some other metric.

Formally,  $G = (V, E, w)$  where  $w: E \rightarrow \mathbb{R}$  maps each edge to a weight.

### 2. Intuition

Think of a **road map** where each road has a distance in kilometres. The roads are edges, cities are vertices, and the distance is the weight.

Another analogy: **Flight routes** where each flight has a ticket price. You want the cheapest route, not just the route with the fewest flights.

#### Step-by-step reasoning:

- Each edge has a cost/value associated with travelling along it.
- Shortest path: find the path with minimum total weight (not minimum number of edges).
- The weight can represent cost, distance, time, profit, or any measurable quantity.
- Negative weights create special cases (possible negative cycles  $\rightarrow$  no shortest path).

### 3. When to Use It

- **Shortest path problems** (Dijkstra's, Bellman-Ford, Floyd-Warshall)

- **Minimum spanning tree** (Kruskal's, Prim's)
- **Network flow** (each edge has a capacity)
- **Resource allocation** (cost/benefit analysis)
- **GPS navigation** (shortest/fastest route)
- **Scheduling with costs**

**Trigger phrases:** "shortest path", "minimum cost", "maximum profit", "weighted", "distance", "capacity", "cheapest", "fastest", "minimum spanning tree", "travelling salesman".

## 4. When Not to Use It

- When all edges have the same cost — use an unweighted graph (BFS is simpler and faster).
- When weights are irrelevant to the problem (e.g., just counting connected components).
- When weights are large and you need integer arithmetic — beware of overflow.
- When negative weights exist and you need shortest paths — Dijkstra's fails; use Bellman-Ford or SPFA.

## 5. Core Concepts

| Concept                        | Explanation                            | Why It Matters                                  |
|--------------------------------|----------------------------------------|-------------------------------------------------|
| <b>Edge weight</b>             | Numerical value assigned to an edge    | Defines cost/distance/capacity of traversal     |
| <b>Path weight</b>             | Sum of weights along a path            | What we minimize/maximize                       |
| <b>Negative weight</b>         | Weight less than 0                     | Breaks Dijkstra's; can create negative cycles   |
| <b>Negative cycle</b>          | A cycle where total weight is negative | No shortest path exists (can keep reducing)     |
| <b>Zero-weight edge</b>        | Edge with weight 0                     | Free traversal; special case in some algorithms |
| <b>Weighted adjacency list</b> | Each entry stores (neighbour, weight)  | Standard representation                         |

## 6. Step-by-Step Algorithm (Representation)

**Adjacency list for weighted graphs:**

1. Create a vector of vector of pairs: `vector<vector<pair<int, int>>> adj(V).`
2. For each edge `(u, v, w)`:
  - `adj[u].push_back({v, w}).`
  - For undirected: also `adj[v].push_back({u, w}).`

**Checking edge weight:**

- Linear scan `adj[u]` to find `v` and return its weight.

## 7. Dry Run

**Input:** Directed weighted graph,  $V = 4$ . Edges:  $(0 \rightarrow 1, 4)$ ,  $(0 \rightarrow 2, 2)$ ,  $(1 \rightarrow 2, 1)$ ,  $(1 \rightarrow 3, 5)$ ,  $(2 \rightarrow 3, 3)$

| Step | Edge                  | adj[0]        | adj[1]        | adj[2]  | adj[3] |
|------|-----------------------|---------------|---------------|---------|--------|
| 1    | $0 \rightarrow 1$ (4) | [(1,4)]       | []            | []      | []     |
| 2    | $0 \rightarrow 2$ (2) | [(1,4),(2,2)] | []            | []      | []     |
| 3    | $1 \rightarrow 2$ (1) | [(1,4),(2,2)] | [(2,1)]       | []      | []     |
| 4    | $1 \rightarrow 3$ (5) | [(1,4),(2,2)] | [(2,1),(3,5)] | []      | []     |
| 5    | $2 \rightarrow 3$ (3) | [(1,4),(2,2)] | [(2,1),(3,5)] | [(3,3)] | []     |

**Query:** Weight of edge  $0 \rightarrow 2$ ? Scan adj[0], find pair (2,2)  $\rightarrow$  weight = 2.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

class WeightedGraph {
private:
    int V;
    vector<vector<pii>> adj;    // each entry: (neighbour, weight)

public:
    WeightedGraph(int vertices) : V(vertices) {
        adj.resize(V);
    }

    // For directed weighted edge
    void addDirectedEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
    }

    // For undirected weighted edge
    void addUndirectedEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    // Get weight of edge u->v, returns -1 if not found
    int getWeight(int u, int v) {
        for (auto [nei, w] : adj[u])
            if (nei == v) return w;
        return -1;
    }

    vector<pii> getNeighbours(int u) { return adj[u]; }
}
```

```

void print() {
    for (int u = 0; u < V; u++) {
        cout << u << " : ";
        for (auto [v, w] : adj[u])
            cout << "(" << v << ", " << w << ") ";
        cout << "\n";
    }
}

};

// Dijkstra's shortest path (single source, non-negative weights)
vector<int> dijkstra(vector<vector<pii>> &adj, int src, int V) {
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

int main() {
    WeightedGraph g(4);
    g.addDirectedEdge(0, 1, 4);
    g.addDirectedEdge(0, 2, 2);
    g.addDirectedEdge(1, 2, 1);
    g.addDirectedEdge(1, 3, 5);
    g.addDirectedEdge(2, 3, 3);
    g.print();

    auto dist = dijkstra(g.getNeighboursRef(), 0, 4);
    for (int i = 0; i < 4; i++)
        cout << "0 -> " << i << " = " << dist[i] << "\n";
    // Output: 0->0=0, 0->1=4, 0->2=2, 0->3=5 (via 0->2->3)
    return 0;
}

```

## 9. Python Implementation

```

import heapq
from typing import List, Tuple

```

```

class WeightedGraph:
    def __init__(self, vertices: int):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_directed_edge(self, u: int, v: int, w: int):
        self.adj[u].append((v, w))

    def add_undirected_edge(self, u: int, v: int, w: int):
        self.adj[u].append((v, w))
        self.adj[v].append((u, w))

    def get_weight(self, u: int, v: int) -> int:
        for nei, w in self.adj[u]:
            if nei == v:
                return w
        return -1

    def get_neighbours(self, u: int) -> List[Tuple[int, int]]:
        return self.adj[u]

    def print_graph(self):
        for u in range(self.V):
            neighbours = ' '.join(f"({v},{w})" for v, w in self.adj[u])
            print(f"{u} : {neighbours}")

def dijkstra(adj: List[List[Tuple[int, int]]], src: int, V: int) -> List[int]:
    dist = [float('inf')] * V
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

# Example
g = WeightedGraph(4)
g.add_directed_edge(0, 1, 4)
g.add_directed_edge(0, 2, 2)
g.add_directed_edge(1, 2, 1)
g.add_directed_edge(1, 3, 5)
g.add_directed_edge(2, 3, 3)
g.print_graph()

dist = dijkstra(g.adj, 0, 4)

```

```
for i in range(4):
    print(f"0 -> {i} = {dist[i]}")
```

## 10. Code Explanation

- **vector<vector<pii>> adj**: Each entry is a list of (neighbour, weight) pairs.
- **addDirectedEdge(u, v, w)**: Pushes (v, w) only to adj[u].
- **addUndirectedEdge(u, v, w)**: Pushes (v, w) to adj[u] and (u, w) to adj[v].
- **Dijkstra's**: Uses a min-heap priority queue. Starts with `dist[src] = 0`. At each step, pops the node with smallest distance and relaxes its neighbours. The `if (d > dist[u]) continue` check is critical — it skips stale entries in the heap.
- **Relaxation**: If `dist[u] + w < dist[v]`, update `dist[v]` and push to heap.

## 11. Complexity Analysis

| Operation                | Complexity          |
|--------------------------|---------------------|
| Add edge                 | $O(1)$              |
| Get weight               | $O(V)$ (adj list)   |
| Dijkstra (binary heap)   | $O((V + E) \log V)$ |
| Bellman-Ford             | $O(V \times E)$     |
| Floyd-Warshall           | $O(V^3)$            |
| Kruskal's MST            | $O(E \log V)$       |
| Prim's MST (binary heap) | $O(E \log V)$       |
| Space (adj list)         | $O(V + E)$          |

## 12. Common Patterns

| Pattern                               | Identification                         | Approach                                                 | Problems                                         |
|---------------------------------------|----------------------------------------|----------------------------------------------------------|--------------------------------------------------|
| <b>Single Source Shortest Path</b>    | "Shortest distance from source"        | Dijkstra (non-negative), Bellman-Ford (negative allowed) | Network Delay Time, Cheapest Flights             |
| <b>All Pairs Shortest Path</b>        | "Shortest distance between every pair" | Floyd-Warshall                                           | Find the City With Smallest Number of Neighbours |
| <b>MST</b>                            | "Minimum cost to connect all nodes"    | Kruskal's (sort edges), Prim's (heap)                    | Min Cost to Connect All Points                   |
| <b>Shortest Path with Constraints</b> | "At most k stops", "with budget"       | Bellman-Ford (k iterations), DP                          | Cheapest Flights Within K Stops                  |

| Pattern                         | Identification                | Approach                            | Problems                      |
|---------------------------------|-------------------------------|-------------------------------------|-------------------------------|
| <b>Maximum Probability Path</b> | "Maximum success probability" | Dijkstra with max-heap (negate log) | Path with Maximum Probability |

## 13. Common Mistakes

- Using Dijkstra's with negative weights — it gives wrong results.
- Using `INT_MAX` for infinity and adding a weight causes overflow.
- Not checking for negative cycles in Bellman-Ford when the problem requires it.
- Forgetting to use `long long` when weights/paths can exceed 32-bit integer range.
- In Dijkstra's, not skipping stale heap entries — leads to  $O(2^V)$  time in worst case.
- In Floyd-Warshall, ordering the loops incorrectly (k must be outermost).

## 14. Edge Cases

- **Single vertex** — distance to itself is 0.
- **Disconnected graph** — unreachable nodes have distance INF.
- **Negative weights** — Dijkstra's fails; use Bellman-Ford.
- **Negative cycles** — no shortest path exists; Bellman-Ford detects them.
- **Zero-weight edges** — handled correctly by all algorithms.
- **Large weights** — risk of overflow; use `long long`.
- **Self-loops** — usually ignored for shortest path ( $\text{dist}[u] = \min(\text{dist}[u], \text{dist}[u] + w) = \text{dist}[u]$  for non-negative).

## 15. Variations

| Variation                | What Changes                      | When Used                                      | Importance      |
|--------------------------|-----------------------------------|------------------------------------------------|-----------------|
| <b>DAG Shortest Path</b> | Topological order + DP relaxation | Faster than Dijkstra for DAGs                  | High            |
| <b>0-1 BFS</b>           | Edge weights are only 0 or 1      | Uses deque instead of heap                     | High (CP)       |
| <b>Dial's algorithm</b>  | Integer weights in small range    | Faster than Dijkstra for small integer weights | Medium          |
| <b>A* search</b>         | Heuristic-guided shortest path    | GPS, pathfinding with heuristics               | Low (not in CP) |

## 16. Related Concepts

- **Unweighted Graph** — Special case where all weights = 1; BFS replaces Dijkstra.
- **Dynamic Programming** — Shortest path in DAG is a DP problem.
- **Minimum Spanning Tree** — Different from shortest path (global connectivity vs single source).
- **Network Flow** — Edge capacities are a type of weight.

## 17. Practice Problems

Easy

- **Network Delay Time** — LeetCode (743) — Dijkstra's
- **Path With Minimum Effort** — LeetCode (1631) — Dijkstra variant

## Medium

- **Cheapest Flights Within K Stops** — LeetCode (787) — Bellman-Ford / Dijkstra with state
- **Minimum Cost to Make at Least One Valid Path** — LeetCode (1368) — 0-1 BFS
- **Find the City With the Smallest Number of Neighbours** — LeetCode (1334) — Floyd-Warshall

## Hard

- **Swim in Rising Water** — LeetCode (778) — Dijkstra / BST with BFS
- **Minimum Cost to Make Array Elements Equal** — LeetCode (2448) — Weighted median
- **Collect Coins in a Tree** — LeetCode (2603) — Tree DP with weighted edges

## 18. Interview Explanation

"A weighted graph stores a numerical value on each edge representing cost, distance, or capacity. I represent it using an adjacency list of pairs (**neighbour**, **weight**). For shortest path with non-negative weights, I use Dijkstra's algorithm with a min-heap priority queue. For negative weights, I use Bellman-Ford, which also detects negative cycles. For all-pairs shortest paths, I use Floyd-Warshall. Time complexity depends on the algorithm: Dijkstra's is  $O((V+E) \log V)$ , Bellman-Ford is  $O(V \times E)$ , and Floyd-Warshall is  $O(V^3)$ ."

## 19. Revision Notes

- Adj list: `vector<vector<pair<int,int>>> adj` — stores (**neighbour**, **weight**).
- Dijkstra's: `priority_queue` min-heap, `dist[src]=0`, skip stale entries.
- Bellman-Ford: relax all edges  $V-1$  times; one more pass detects negative cycles.
- Floyd-Warshall:  $k$  is outermost loop, `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`.
- 0-1 BFS: use `deque`, push front for weight 0, push back for weight 1.
- DAG shortest path: topological sort + DP in  $O(V+E)$ .

## 20. Final Cheat Sheet

| Aspect                | Summary                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------|
| <b>When to use</b>    | Edge costs matter (shortest path, MST, flow)                                               |
| <b>Representation</b> | <code>vector&lt;vector&lt;pair&lt;int,int&gt;&gt;&gt; adj</code>                           |
| <b>Dijkstra</b>       | $O((V+E) \log V)$ , non-negative weights only                                              |
| <b>Bellman-Ford</b>   | $O(V \times E)$ , handles negative weights, detects negative cycles                        |
| <b>Floyd-Warshall</b> | $O(V^3)$ , all-pairs shortest paths                                                        |
| <b>Key traps</b>      | Overflow, negative weights with Dijkstra, stale heap entries, loop order in Floyd-Warshall |
| <b>Must know</b>      | Dijkstra, Bellman-Ford, Floyd-Warshall, 0-1 BFS                                            |

# 4. Unweighted Graph

## 1. Overview

An **unweighted graph** is a graph where edges have no associated weight. All edges are treated equally — the cost of traversing any edge is 1 (or 1 unit).

This is the simplest form of a graph. It is the default assumption when no weights are mentioned.

## 2. Intuition

Think of a **social network** where you just care about friendships, not the strength of the friendship. Two people are either friends or they are not — there's no "cost" to the friendship.

Another analogy: **Hyperlinks between web pages**. A link either exists or it doesn't. There's no cost to clicking a link.

### Step-by-step reasoning:

- Every edge is identical in cost.
- The "shortest path" is the path with the fewest edges.
- BFS finds the shortest path in  $O(V+E)$  — simpler and faster than Dijkstra's.
- No weights means no need for priority queues or complex data structures.

## 3. When to Use It

- When the problem doesn't mention any weights/costs on edges.
- When you only care about the number of edges in a path.
- When you need the shortest path and all edges are equally expensive.
- For graph traversal (BFS/DFS) where weights don't matter.
- For finding connected components, bipartiteness, cycles.

**Trigger phrases:** "find the shortest path", "minimum number of edges", "level order", "distance in terms of edges", "unweighted", "friendship network", "connections".

## 4. When Not to Use It

- When edges have different costs — use a weighted graph.
- When the problem explicitly mentions weights, distances, or costs.
- When you need MST (minimum spanning tree) — weights are required.
- When you need to minimize something other than edge count (money, time, etc.).

## 5. Core Concepts

| Concept             | Explanation                        | Why It Matters                            |
|---------------------|------------------------------------|-------------------------------------------|
| Edge count distance | Distance = number of edges in path | BFS finds shortest path in terms of edges |

| Concept     | Explanation                                 | Why It Matters                                |
|-------------|---------------------------------------------|-----------------------------------------------|
| Level/Depth | Distance from source in BFS                 | Used for level-order traversal, shortest path |
| BFS tree    | Tree formed by BFS parent pointers          | Gives shortest path from source to all nodes  |
| Diameter    | Longest shortest path between any two nodes | Found by two BFS runs                         |

## 6. Step-by-Step Algorithm (BFS Shortest Path in Unweighted Graph)

1. Initialize `dist[]` array with INF, `dist[src] = 0`.
2. Create a queue, push `src`.
3. While queue is not empty:
  - Pop front `u`.
  - For each neighbour `v` of `u`:
    - If `dist[v]` is INF, set `dist[v] = dist[u] + 1`, push `v` to queue.
4. Return `dist[]`.

## 7. Dry Run

**Input:** V = 5, edges = [(0,1), (0,2), (1,3), (2,3), (3,4)], source = 0.

| Step | Queue                           | dist[0] | dist[1] | dist[2] | dist[3] | dist[4] |
|------|---------------------------------|---------|---------|---------|---------|---------|
| 0    | [0]                             | 0       | INF     | INF     | INF     | INF     |
| 1    | Pop 0, push 1,2                 | 0       | 1       | 1       | INF     | INF     |
| 2    | Pop 1, push 3                   | 0       | 1       | 1       | 2       | INF     |
| 3    | Pop 2, push (3 already visited) | 0       | 1       | 1       | 2       | INF     |
| 4    | Pop 3, push 4                   | 0       | 1       | 1       | 2       | 3       |
| 5    | Pop 4                           | 0       | 1       | 1       | 2       | 3       |

**Result:** Shortest distances from 0: 0→0=0, 0→1=1, 0→2=1, 0→3=2, 0→4=3.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class UnweightedGraph {
private:
    int V;
    vector<vector<int>>> adj;

public:
```

```

UnweightedGraph(int vertices) : V(vertices) {
    adj.resize(V);
}

void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u); // remove for directed
}

// BFS shortest path (in terms of number of edges)
vector<int> shortestPath(int src) {
    vector<int> dist(V, -1);
    queue<int> q;
    dist[src] = 0;
    q.push(src);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}

// Level-order traversal
vector<vector<int>> levelOrder(int src) {
    vector<vector<int>> levels;
    vector<bool> visited(V, false);
    queue<int> q;
    visited[src] = true;
    q.push(src);

    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        while (sz-- > 0) {
            int u = q.front(); q.pop();
            level.push_back(u);
            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        levels.push_back(level);
    }
    return levels;
}

```

```

// Compute graph diameter (longest shortest path)
pair<int, int> farthestNode(int src) {
    auto dist = shortestPath(src);
    int farNode = src;
    for (int i = 0; i < V; i++)
        if (dist[i] > dist[farNode]) farNode = i;
    return {farNode, dist[farNode]};
}

int diameter() {
    auto [far, _] = farthestNode(0);
    auto [_, d] = farthestNode(far);
    return d;
}
};

int main() {
    UnweightedGraph g(5);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 3);
    g.addEdge(3, 4);

    auto dist = g.shortestPath(0);
    for (int i = 0; i < 5; i++)
        cout << "0 -> " << i << " = " << dist[i] << "\n";

    cout << "Diameter: " << g.diameter() << "\n"; // 3 (0→1→3→4 or 0→2→3→4)
    return 0;
}

```

## 9. Python Implementation

```

from collections import deque
from typing import List, Tuple

class UnweightedGraph:
    def __init__(self, vertices: int):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_edge(self, u: int, v: int):
        self.adj[u].append(v)
        self.adj[v].append(u) # remove for directed

    def shortest_path(self, src: int) -> List[int]:
        dist = [-1] * self.V
        dist[src] = 0
        q = deque([src])

```

```

    while q:
        u = q.popleft()
        for v in self.adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

def level_order(self, src: int) -> List[List[int]]:
    levels = []
    visited = [False] * self.V
    q = deque([src])
    visited[src] = True

    while q:
        level = []
        for _ in range(len(q)):
            u = q.popleft()
            level.append(u)
            for v in self.adj[u]:
                if not visited[v]:
                    visited[v] = True
                    q.append(v)
        levels.append(level)
    return levels

def _farthest_node(self, src: int) -> Tuple[int, int]:
    dist = self.shortest_path(src)
    far = max(range(self.V), key=lambda i: dist[i])
    return far, dist[far]

def diameter(self) -> int:
    far, _ = self._farthest_node(0)
    _, d = self._farthest_node(far)
    return d

# Example
g = UnweightedGraph(5)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(2, 3)
g.add_edge(3, 4)

dist = g.shortest_path(0)
for i in range(5):
    print(f"0 -> {i} = {dist[i]}")
print(f"Diameter: {g.diameter()}")

```

## 10. Code Explanation

- **shortestPath(src)**: BFS from source. `dist[v] = -1` means unvisited. When we first visit a node, its distance is `dist[u] + 1`. Since BFS processes nodes in order of increasing distance, the first visit gives the shortest path.
- **levelOrder(src)**: Processes nodes level by level. Uses a `for` loop sized by `q.size()` to capture one level at a time.
- **farthestNode(src)**: Finds the node farthest from `src` and its distance.
- **diameter()**: Two BFS runs: first to find one endpoint of the diameter, second to find the actual diameter length.

## 11. Complexity Analysis

| Operation         | Complexity                |
|-------------------|---------------------------|
| Add edge          | $O(1)$                    |
| BFS shortest path | $O(V + E)$                |
| Level order       | $O(V + E)$                |
| Diameter          | $O(V + E)$ (two BFS runs) |
| Space             | $O(V + E)$                |

## 12. Common Patterns

| Pattern                                  | Identification                       | Approach                     | Problems                       |
|------------------------------------------|--------------------------------------|------------------------------|--------------------------------|
| <b>Shortest Path in Unweighted Graph</b> | "Minimum number of edges"            | BFS                          | Shortest Path in Binary Matrix |
| <b>Level Order / Minimum Steps</b>       | "Minimum moves", "knight moves"      | BFS with level tracking      | Knight Moves, Word Ladder      |
| <b>Graph Diameter</b>                    | "Longest path", "maximum distance"   | Two BFS runs                 | Tree Diameter                  |
| <b>Bipartite Check</b>                   | 2-coloring                           | BFS with alternating colours | Is Graph Bipartite?            |
| <b>Nearest Distance from Source Set</b>  | "Nearest 0", "distance to nearest 1" | Multi-source BFS             | 01 Matrix, Rotting Oranges     |

## 13. Common Mistakes

- Using DFS instead of BFS for shortest path — DFS doesn't guarantee shortest path.
- Forgetting to mark visited when pushing to queue (not when popping) — leads to multiple pushes of the same node.
- Not handling disconnected components — BFS only visits the component containing the source.
- Using recursion for BFS (BFS is iterative by nature).
- Confusing `-1` (unvisited) with actual distance 0.

## 14. Edge Cases

- **Single vertex** — distance to itself = 0.
- **Disconnected graph** — some distances remain INF/-1.
- **Empty graph** (no edges) — all distances INF except source.
- **Complete graph** — all distances = 1 (except source = 0).
- **Graph with self-loops** — BFS handles them correctly ( $\text{dist}[u] = 0$ , self-loop gives  $\text{dist}[u] + 1 = 1$ , which is  $> 0$ , so skipped).

## 15. Variations

| Variation               | What Changes                            | When Used                           | Importance |
|-------------------------|-----------------------------------------|-------------------------------------|------------|
| <b>Multi-source BFS</b> | Multiple starting nodes                 | Finding nearest distance from a set | High       |
| <b>0-1 BFS</b>          | Edge weights are 0 or 1                 | Deque-based BFS for near-unweighted | High       |
| <b>BFS on grid</b>      | 4-directional or 8-directional movement | Maze, grid problems                 | Very High  |

## 16. Related Concepts

- **Weighted Graph** — Unweighted is a special case of weighted where all weights = 1.
- **BFS** — The fundamental traversal algorithm for unweighted graphs.
- **DFS** — Alternative traversal, but doesn't give shortest path.
- **Dynamic Programming** — BFS on unweighted DAG is equivalent to DP.

## 17. Practice Problems

### Easy

- **Shortest Path in Binary Matrix** — LeetCode (1091) — BFS on grid
- **Nearest Exit from Entrance in Maze** — LeetCode (1926) — BFS on grid

### Medium

- **Word Ladder** — LeetCode (127) — BFS on implicit graph
- **01 Matrix** — LeetCode (542) — Multi-source BFS
- **Rotting Oranges** — LeetCode (994) — Multi-source BFS

### Hard

- **Minimum Knight Moves** — LeetCode (1197) — BFS with pruning
- **Shortest Path Visiting All Nodes** — LeetCode (847) — BFS + bitmask DP
- **Snakes and Ladders** — LeetCode (909) — BFS on board

## 18. Interview Explanation

"An unweighted graph is the simplest graph model where all edges have equal cost. I use BFS to find the shortest path in terms of edge count in  $O(V+E)$  time. The key insight is that BFS processes nodes in

order of increasing distance from the source, so the first time we visit a node during BFS, it's via the shortest path. I use a queue, a visited array, and a distance array. For problems like nearest distance from a set of sources, I use multi-source BFS."

## 19. Revision Notes

- BFS uses queue, DFS uses stack (or recursion).
- BFS gives shortest path in unweighted graphs.
- First visit = shortest distance (BFS property).
- Mark visited when pushing to queue, not when popping.
- $\text{dist}[v] = \text{dist}[u] + 1$  — simple and correct.
- Multi-source BFS: push all sources initially with  $\text{dist}=0$ .
- $\text{diameter} = 2 \times \text{BFS}$ .

## 20. Final Cheat Sheet

| Aspect             | Summary                                                       |
|--------------------|---------------------------------------------------------------|
| <b>When to use</b> | All edges have equal cost; minimize edge count                |
| <b>Algorithm</b>   | BFS with queue and distance array                             |
| <b>Complexity</b>  | $O(V + E)$ time, $O(V)$ space                                 |
| <b>Key idea</b>    | First visit = shortest path                                   |
| <b>Key traps</b>   | Mark visited on push, not pop; handle disconnected components |
| <b>Must know</b>   | BFS, level order, multi-source BFS, diameter                  |

## 5. Adjacency Matrix

### 1. Overview

An **adjacency matrix** is a 2D array of size  $V \times V$  where  $\text{mat}[i][j]$  stores information about the edge from vertex  $i$  to vertex  $j$ .

- For unweighted graphs:  $\text{mat}[i][j] = 1$  if edge exists,  $0$  otherwise.
- For weighted graphs:  $\text{mat}[i][j] = \text{weight}$  of edge,  $\text{INF}$  or  $0$  if no edge.
- For undirected graphs:  $\text{mat}[i][j] = \text{mat}[j][i]$ .

### 2. Intuition

Think of a **spreadsheet** where rows are source vertices, columns are destination vertices. Cell  $(\text{row}, \text{col})$  tells you if there's a direct connection from row to col.

Another analogy: A **flight booking table** where rows are departure cities, columns are arrival cities, and each cell shows the flight price. You can instantly check if a direct flight exists between any two cities.

**Step-by-step reasoning:**

- `mat[u][v]` = edge status/weight between u and v.
- Checking if an edge exists is  $O(1)$  — just look up the cell.
- Adding/removing an edge is  $O(1)$  — just set the cell value.
- Space is  $O(V^2)$  — this is the main limitation.
- Iterating over all neighbours of a vertex takes  $O(V)$  time (you must scan the entire row).

### 3. When to Use It

- When  $V$  is small (typically  $V \leq 1000$ , preferably  $V \leq 500$ ).
- When you need  **$O(1)$  edge existence checks** frequently.
- When you need to **update edge weights** frequently.
- For algorithms like **Floyd-Warshall** (all-pairs shortest path) that inherently use a matrix.
- When the graph is **dense** ( $E \approx V^2$ ).
- When you need to quickly check if a particular edge exists in a **directed graph**.

**Trigger phrases:** "dense graph", "complete graph", "all pairs", "Floyd-Warshall", "edge lookup", "frequent edge queries".

### 4. When Not to Use It

- When  $V$  is large ( $V > 10^4$ ) —  $O(V^2)$  memory is too expensive.
- For **sparse graphs** ( $E \ll V^2$ ) — adjacency list is more memory-efficient.
- When you need to iterate over all neighbours often —  $O(V)$  per vertex vs  $O(\text{degree})$  with adjacency list.
- When memory is constrained.
- When you need to add/remove vertices dynamically — resizing the matrix is expensive.

### 5. Core Concepts

| Concept    | Explanation                                                        | Why It Matters                              |
|------------|--------------------------------------------------------------------|---------------------------------------------|
| Cell value | <code>mat[i][j]</code> = 1/weight (edge exists) or 0/INF (no edge) | Defines the graph                           |
| Symmetry   | Undirected: <code>mat[i][j] == mat[j][i]</code>                    | Reduces storage by half if optimized        |
| Row        | All outgoing edges from vertex i                                   | Iterating over row = getting all neighbours |
| Column     | All incoming edges to vertex j                                     | In-degree information without extra work    |
| Diagonal   | <code>mat[i][i]</code> = self-loop                                 | Usually 0 unless self-loops are allowed     |

### 6. Step-by-Step Algorithm (Building and Operations)

**Building:**

1. Create  $V \times V$  matrix initialized to 0 (or INF for weighted).
2. For each edge  $(u, v)$ :
  - Unweighted: `mat[u][v] = 1`.
  - Weighted: `mat[u][v] = w`.

- Undirected: also  $\text{mat}[v][u] = 1$  (or  $w$ ).

### Operations:

- Check edge:  $\text{mat}[u][v] \neq 0$  —  $O(1)$ .
- Add edge:  $\text{mat}[u][v] = 1$  (or weight) —  $O(1)$ .
- Remove edge:  $\text{mat}[u][v] = 0$  —  $O(1)$ .
- Get all neighbours: scan row  $u$  —  $O(V)$ .
- Get in-degree: scan column  $u$  —  $O(V)$ .

## 7. Dry Run

**Input:**  $V = 4$ , directed edges =  $[(0,1), (0,2), (1,2), (2,3), (3,1)]$

Initial matrix ( $4 \times 4$ , all zeros):

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 0 |

After adding edges:

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 2 | 0 | 0 | 0 | 1 |
| 3 | 0 | 1 | 0 | 0 |

### Queries:

- Edge  $0 \rightarrow 1$ ?  $\text{mat}[0][1] = 1 \rightarrow \text{Yes}$ .
- Edge  $1 \rightarrow 0$ ?  $\text{mat}[1][0] = 0 \rightarrow \text{No}$ .
- Neighbours of 0? Row 0:  $[1, 1, 0, 0] \rightarrow \{1, 2\}$ .
- In-degree of 1? Column 1:  $[1, 0, 0, 1] \rightarrow 2$  (from 0 and 3).

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class AdjacencyMatrix {
private:
    int V;
```

```
vector<vector<int>> mat;

public:
    AdjacencyMatrix(int vertices) : V(vertices) {
        mat.assign(V, vector<int>(V, 0));
    }

    // For weighted graph, pass weight w
    void addEdge(int u, int v, int w = 1) {
        mat[u][v] = w;
    }

    void addUndirectedEdge(int u, int v, int w = 1) {
        mat[u][v] = w;
        mat[v][u] = w;
    }

    bool hasEdge(int u, int v) {
        return mat[u][v] != 0;
    }

    int getWeight(int u, int v) {
        return mat[u][v];
    }

    void removeEdge(int u, int v) {
        mat[u][v] = 0;
    }

    vector<int> getNeighbours(int u) {
        vector<int> neighbours;
        for (int v = 0; v < V; v++)
            if (mat[u][v] != 0)
                neighbours.push_back(v);
        return neighbours;
    }

    int inDegree(int u) {
        int deg = 0;
        for (int i = 0; i < V; i++)
            if (mat[i][u] != 0) deg++;
        return deg;
    }

    int outDegree(int u) {
        return getNeighbours(u).size();
    }

    void print() {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++)
                cout << mat[i][j] << " ";
            cout << "\n";
        }
    }
```

```

    }
};

// Floyd-Warshall all-pairs shortest path
vector<vector<int>> floydWarshall(vector<vector<int>> &graph) {
    int V = graph.size();
    vector<vector<int>> dist = graph;

    for (int k = 0; k < V; k++)
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

    return dist;
}

int main() {
    AdjacencyMatrix g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 3);
    g.addEdge(3, 1);
    g.print();
    // 0 1 1 0
    // 0 0 1 0
    // 0 0 0 1
    // 0 1 0 0

    cout << "Has 0->1: " << g.hasEdge(0, 1) << "\n"; // 1
    cout << "Has 1->0: " << g.hasEdge(1, 0) << "\n"; // 0
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class AdjacencyMatrix:
    def __init__(self, vertices: int):
        self.V = vertices
        self.mat = [[0] * vertices for _ in range(vertices)]

    def add_edge(self, u: int, v: int, w: int = 1):
        self.mat[u][v] = w

    def add_undirected_edge(self, u: int, v: int, w: int = 1):
        self.mat[u][v] = w
        self.mat[v][u] = w

```

```

def has_edge(self, u: int, v: int) -> bool:
    return self.mat[u][v] != 0

def get_weight(self, u: int, v: int) -> int:
    return self.mat[u][v]

def remove_edge(self, u: int, v: int):
    self.mat[u][v] = 0

def get_neighbours(self, u: int) -> List[int]:
    return [v for v in range(self.V) if self.mat[u][v] != 0]

def in_degree(self, u: int) -> int:
    return sum(1 for i in range(self.V) if self.mat[i][u] != 0)

def out_degree(self, u: int) -> int:
    return len(self.get_neighbours(u))

def print_matrix(self):
    for row in self.mat:
        print(' '.join(map(str, row)))

def floyd_warshall(graph: List[List[int]]) -> List[List[int]]:
    V = len(graph)
    INF = float('inf')
    dist = [row[:] for row in graph]

    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist

# Example
g = AdjacencyMatrix(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)
g.add_edge(3, 1)
g.print_matrix()

```

## 10. Code Explanation

- `mat.assign(V, vector<int>(V, 0))`: Creates a  $V \times V$  matrix filled with 0.
- `addEdge(u, v, w)`: Sets `mat[u][v] = w`. Default weight is 1 for unweighted graphs.
- `hasEdge(u, v)`: Returns `mat[u][v] != 0`.  $O(1)$  — the main advantage of adjacency matrix.
- `getNeighbours(u)`: Scans the entire row of size  $V$  —  $O(V)$ . This is slower than adjacency list for sparse graphs.

- **inDegree(u)**: Scans the entire column of size  $V$  —  $O(V)$ . This is a nice feature: column scan gives in-degree directly without extra data structures.
- **Floyd-Warshall**: Uses the matrix representation naturally. The triple nested loop computes all-pairs shortest paths.

## 11. Complexity Analysis

| Operation          | Complexity | Notes                   |
|--------------------|------------|-------------------------|
| Add edge           | $O(1)$     | Direct index assignment |
| Remove edge        | $O(1)$     | Direct index assignment |
| Check edge         | $O(1)$     | The main advantage      |
| Get all neighbours | $O(V)$     | Must scan entire row    |
| In-degree          | $O(V)$     | Scan column             |
| Out-degree         | $O(V)$     | Scan row                |
| Space              | $O(V^2)$   | The main disadvantage   |
| Floyd-Warshall     | $O(V^3)$   | Inherently uses matrix  |

## 12. Common Patterns

| Pattern                        | Identification                         | Approach                                  | Problems                                  |
|--------------------------------|----------------------------------------|-------------------------------------------|-------------------------------------------|
| <b>All-Pairs Shortest Path</b> | "Shortest distance between every pair" | Floyd-Warshall                            | Find the City With Smallest Neighbourhood |
| <b>Transitive Closure</b>      | "Can you reach from $i$ to $j$ "       | Floyd-Warshall variant (boolean)          | Reachability in directed graph            |
| <b>Dense Graph Problems</b>    | $E \approx V^2$ , many edge queries    | Use matrix for $O(1)$ lookups             | Graph connectivity queries                |
| <b>Graph Powering</b>          | Paths of length $k$                    | Matrix exponentiation on adjacency matrix | Number of paths of length $k$             |

## 13. Common Mistakes

- Using `int` for INF and adding two INF values causes overflow. Use `INT_MAX/2` or check for INF before adding.
- Forgetting to initialize `mat[i][i] = 0` for shortest path algorithms.
- Using adjacency matrix when  $V$  is too large (e.g.,  $V = 10^5$ ) — memory blowup.
- Confusing `mat[i][j]` and `mat[j][i]` in directed graphs.
- In Floyd-Warshall, putting `k` loop innermost — it must be the outermost loop for correctness.
- Using adjacency matrix for sparse graphs and iterating over all neighbours frequently —  $O(V^2)$  total vs  $O(E)$  with adjacency list.

## 14. Edge Cases

- **V = 0** — matrix is empty.
- **V = 1** — single cell; self-loop if present.
- **Complete graph** — all cells = 1 (except diagonal).
- **Disconnected graph** — many zeros in the matrix.
- **Weighted graph** — cells contain weights, not just 0/1.
- **Self-loops** — diagonal elements.

## 15. Variations

| Variation               | What Changes                                        | When Used                     | Importance                 |
|-------------------------|-----------------------------------------------------|-------------------------------|----------------------------|
| <b>Boolean matrix</b>   | Store <code>bool</code> instead of <code>int</code> | Memory savings for unweighted | Medium                     |
| <b>Bit matrix</b>       | Use bitset for each row                             | Further memory optimization   | Low (CP only)              |
| <b>Sparse matrix</b>    | Store only non-zero entries                         | Very large V, very sparse     | Low (use adj list instead) |
| <b>Incidence matrix</b> | $V \times E$ matrix, rows=vertices, cols=edges      | Hypergraphs, theory           | Rare                       |

## 16. Related Concepts

- **Adjacency List** — The alternative representation; better for sparse graphs, neighbour iteration.
- **Edge List** — List of `(u, v, w)` tuples; used in Kruskal's algorithm.
- **Floyd-Warshall** — The algorithm that inherently needs a matrix.
- **Matrix Exponentiation** — Used to count paths of length k in a graph.

## 17. Practice Problems

### Easy

- **Find if Path Exists in Graph** — LeetCode (1971) — Can use matrix for small V
- **Degree of a Vertex** — GFG — Matrix query practice

### Medium

- **Find the City With the Smallest Number of Neighbours at a Threshold Distance** — LeetCode (1334) — Floyd-Warshall
- **Number of Provinces** — LeetCode (547) — Matrix adjacency + DFS

### Hard

- **Minimum Cost to Make at Least One Valid Path** — LeetCode (1368) — 0-1 BFS on grid (matrix)
- **Number of Good Paths** — LeetCode (2421) — DSU on matrix-like graph

## 18. Interview Explanation

"An adjacency matrix is a  $V \times V$  2D array where  $\text{mat}[i][j]$  indicates whether an edge exists from  $i$  to  $j$ . It gives  $O(1)$  edge existence checks and updates, which is great for dense graphs. The main drawback is  $O(V^2)$  space, which makes it impractical for  $V > 10^4$ . I use it when the graph is dense or when I need Floyd-Warshall for all-pairs shortest paths. For sparse graphs, I prefer adjacency lists."

## 19. Revision Notes

- $O(1)$  edge lookup,  $O(V^2)$  space.
- Use for dense graphs or when  $V$  is small ( $\leq 1000$ ).
- `mat[i][j] = 1` (or weight) means edge  $i \rightarrow j$  exists.
- Undirected: `mat[i][j] = mat[j][i]`.
- Floyd-Warshall:  $k$  outermost loop.
- `INF = INT_MAX/2` to avoid overflow.
- Row scan = neighbours, column scan = in-degree.

## 20. Final Cheat Sheet

| Aspect           | Summary                                                                      |
|------------------|------------------------------------------------------------------------------|
| When to use      | Dense graphs, small $V$ , frequent edge lookups, Floyd-Warshall              |
| Structure        | <code>vector&lt;vector&lt;int&gt;&gt; mat(V, vector&lt;int&gt;(V, 0))</code> |
| Add/Check/Remove | $O(1)$ — <code>mat[u][v] = w</code>                                          |
| Neighbours       | $O(V)$ — scan row                                                            |
| Space            | $O(V^2)$ — main limitation                                                   |
| Key traps        | INF overflow, Floyd-Warshall $k$ -order, memory for large $V$                |
| Must know        | $O(1)$ lookup, Floyd-Warshall, dense graph use case                          |

## 6. Adjacency List

### 1. Overview

An **adjacency list** represents a graph as an array of lists. Each vertex has a list of its neighbours (and edge weights, if applicable). It is the most common graph representation in competitive programming and interviews.

### 2. Intuition

Think of a **phone book** where each person has a list of their friends. If you want to know who someone is connected to, you just look at their list.

Another analogy: **Instagram followers**. Each user has a list of accounts they follow. To see who someone follows, you look at their following list.

**Step-by-step reasoning:**

- Space =  $O(V + E)$  — proportional to actual edges, not  $V^2$ .
- Checking if an edge exists between  $u$  and  $v$  requires scanning  $u$ 's list —  $O(\text{degree}(u))$ .
- Iterating over all neighbours of  $u$  is  $O(\text{degree}(u))$  — optimal.
- Adding an edge is  $O(1)$  — just append to the list.
- This is the **default choice** for most graph problems.

### 3. When to Use It

- **Almost always** — adjacency list is the default graph representation.
- For **sparse graphs** ( $E \ll V^2$ ) — which is most real-world graphs.
- When you need to **iterate over neighbours** frequently (BFS, DFS, Dijkstra).
- When  $V$  is large (up to  $10^5$  or  $10^6$ ).
- When you need to **add edges dynamically**.
- For **competitive programming** — it's the standard representation.

**Trigger phrases:** "graph", "tree", "adjacency", "neighbours", "BFS", "DFS", "undirected", "directed" — adjacency list is the default.

### 4. When Not to Use It

- When you need  **$O(1)$  edge existence checks** — adjacency matrix is better.
- When the graph is **dense** ( $E \approx V^2$ ) — matrix may be simpler and equally efficient.
- When you need to **frequently delete edges** — deletion from a vector is  $O(V)$  (use `unordered_set` or `list` instead).
- When you need to check if a specific edge exists many times — use adjacency matrix or `unordered_set` per vertex.

### 5. Core Concepts

| Concept                          | Explanation                                                        | Why It Matters                                          |
|----------------------------------|--------------------------------------------------------------------|---------------------------------------------------------|
| <b>Vector of vectors</b>         | <code>vector&lt;vector&lt;int&gt;&gt; adj</code> — basic structure | Each <code>adj[u]</code> is a list of neighbours of $u$ |
| <b>Degree = list size</b>        | <code>adj[u].size()</code> = degree of $u$                         | $O(1)$ degree query                                     |
| <b>Pairs for weighted</b>        | <code>vector&lt;vector&lt;pair&lt;int,int&gt;&gt;&gt;</code>       | Stores (neighbour, weight)                              |
| <b>Undirected double storage</b> | Edge stored in both lists                                          | Space = $2E$ instead of $E$                             |
| <b>Hash set per vertex</b>       | <code>vector&lt;unordered_set&lt;int&gt;&gt;</code>                | $O(1)$ edge lookup, more memory                         |

### 6. Step-by-Step Algorithm (Building)

1. Create `vector<vector<int>> adj(V)`.
2. For each edge  $(u, v)$ :
  - **Directed:** `adj[u].push_back(v)`.
  - **Undirected:** `adj[u].push_back(v)` and `adj[v].push_back(u)`.

3. Optionally sort each adjacency list if order matters.

## 7. Dry Run

**Input:**  $V = 5$ , undirected edges = [(0,1), (0,4), (1,2), (1,3), (1,4), (2,3), (3,4)]

| Vertex | Adjacency List |
|--------|----------------|
| 0      | [1, 4]         |
| 1      | [0, 2, 3, 4]   |
| 2      | [1, 3]         |
| 3      | [1, 2, 4]      |
| 4      | [0, 1, 3]      |

### Operations:

- Degree of 1: `adj[1].size() = 4`.
- Neighbours of 2: `adj[2] = [1, 3]`.
- Edge 0→4? Scan `adj[0]` → found.
- Edge 2→4? Scan `adj[2]` → not found.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class AdjacencyList {
private:
    int V;
    vector<vector<int>> adj;           // unweighted
    vector<vector<pair<int,int>>> wadj; // weighted

public:
    AdjacencyList(int vertices) : V(vertices) {
        adj.resize(V);
        wadj.resize(V);
    }

    // Unweighted
    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    void addUndirectedEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // Weighted
```

```
void addWeightedEdge(int u, int v, int w) {
    wadj[u].push_back({v, w});
}

void addWeightedUndirected(int u, int v, int w) {
    wadj[u].push_back({v, w});
    wadj[v].push_back({u, w});
}

// Check if edge exists (linear scan)
bool hasEdge(int u, int v) {
    for (int x : adj[u])
        if (x == v) return true;
    return false;
}

// Get neighbours
vector<int> getNeighbours(int u) { return adj[u]; }

// Degree
int degree(int u) { return adj[u].size(); }

// Remove duplicates and sort (optional, for cleaner output)
void normalize() {
    for (int u = 0; u < V; u++) {
        sort(adj[u].begin(), adj[u].end());
        adj[u].erase(unique(adj[u].begin(), adj[u].end()), adj[u].end());
    }
}

// BFS using adjacency list
vector<int> bfs(int src) {
    vector<int> dist(V, -1);
    queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}

void print() {
    for (int u = 0; u < V; u++) {
        cout << u << " -> ";
        for (int v : adj[u])
            cout << v << " ";
        cout << "\n";
    }
}
```

```

    }
}
};

int main() {
    AdjacencyList g(5);
    g.addUndirectedEdge(0, 1);
    g.addUndirectedEdge(0, 4);
    g.addUndirectedEdge(1, 2);
    g.addUndirectedEdge(1, 3);
    g.addUndirectedEdge(1, 4);
    g.addUndirectedEdge(2, 3);
    g.addUndirectedEdge(3, 4);
    g.print();
    // 0 -> 1 4
    // 1 -> 0 2 3 4
    // 2 -> 1 3
    // 3 -> 1 2 4
    // 4 -> 0 1 3

    auto dist = g.bfs(0);
    for (int i = 0; i < 5; i++)
        cout << "0->" << i << " = " << dist[i] << "\n";
    return 0;
}

```

## 9. Python Implementation

```

from collections import deque
from typing import List, Tuple

class AdjacencyList:
    def __init__(self, vertices: int):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]
        self.wadj = [[] for _ in range(vertices)]

    def add_edge(self, u: int, v: int):
        self.adj[u].append(v)

    def add_undirected_edge(self, u: int, v: int):
        self.adj[u].append(v)
        self.adj[v].append(u)

    def add_weighted_edge(self, u: int, v: int, w: int):
        self.wadj[u].append((v, w))

    def add_weighted_undirected(self, u: int, v: int, w: int):
        self.wadj[u].append((v, w))
        self.wadj[v].append((u, w))

```

```

def has_edge(self, u: int, v: int) -> bool:
    return v in self.adj[u]

def get_neighbours(self, u: int) -> List[int]:
    return self.adj[u]

def degree(self, u: int) -> int:
    return len(self.adj[u])

def normalize(self):
    for u in range(self.V):
        self.adj[u] = sorted(set(self.adj[u]))

def bfs(self, src: int) -> List[int]:
    dist = [-1] * self.V
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in self.adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

def print_graph(self):
    for u in range(self.V):
        print(f"{u} -> {' '.join(map(str, self.adj[u]))}")

# Example
g = AdjacencyList(5)
g.add_undirected_edge(0, 1)
g.add_undirected_edge(0, 4)
g.add_undirected_edge(1, 2)
g.add_undirected_edge(1, 3)
g.add_undirected_edge(1, 4)
g.add_undirected_edge(2, 3)
g.add_undirected_edge(3, 4)
g.print_graph()

```

## 10. Code Explanation

- **vector<vector<int>> adj**: An array of V vectors. **adj[u]** stores all neighbours of vertex u.
- **addUndirectedEdge(u, v)**: Adds v to adj[u] and u to adj[v]. This doubles the storage but makes traversal natural.
- **hasEdge(u, v)**: Linear scan of **adj[u]**. For O(1) lookups, replace with **unordered\_set<int>** per vertex.
- **degree(u)**: Returns **adj[u].size()**. For undirected graphs, this is the correct degree.
- **normalize()**: Sorts and removes duplicates. Useful when edges can be added multiple times.
- **bfs(src)**: Standard BFS using a queue. O(V+E) time.

## 11. Complexity Analysis

| Operation          | Adjacency List        | Adjacency Matrix |
|--------------------|-----------------------|------------------|
| Add edge           | $O(1)$                | $O(1)$           |
| Remove edge        | $O(V)$                | $O(1)$           |
| Check edge         | $O(\text{degree}(u))$ | $O(1)$           |
| Iterate neighbours | $O(\text{degree}(u))$ | $O(V)$           |
| Space              | $O(V + E)$            | $O(V^2)$         |

## 12. Common Patterns

| Pattern                     | Identification                 | Approach                                 | Problems                  |
|-----------------------------|--------------------------------|------------------------------------------|---------------------------|
| <b>Graph Traversal</b>      | "Visit all nodes", "find path" | BFS/DFS on adjacency list                | Almost all graph problems |
| <b>Shortest Path</b>        | "Minimum distance"             | BFS (unweighted), Dijkstra (weighted)    | Network Delay Time        |
| <b>Cycle Detection</b>      | "Detect cycle"                 | DFS with parent/recStack                 | Course Schedule           |
| <b>Topological Sort</b>     | "Ordering with dependencies"   | Kahn's algorithm (BFS) or DFS            | Course Schedule II        |
| <b>Connected Components</b> | "Count groups"                 | Iterate DFS/BFS from each unvisited node | Number of Provinces       |

## 13. Common Mistakes

- Forgetting to add the reverse edge for undirected graphs.
- Using `adj[u][v]` instead of iterating (vector doesn't have arbitrary indexing by neighbour).
- Using adjacency list when the graph is dense and you need many edge existence checks.
- Not clearing the adjacency list between test cases in CP.
- Passing adjacency list by value to functions ( $O(V+E)$  copy) instead of by reference.
- Not handling duplicate edges (multiple calls to `addEdge` with same pair).

## 14. Edge Cases

- **Single vertex** — adjacency list is empty.
- **Disconnected graph** — some vertices have degree 0.
- **Complete graph** — each vertex has  $V-1$  neighbours.
- **Graph with self-loops** — `adj[u]` contains `u`.
- **Graph with parallel edges** — multiple entries of same neighbour.
- **Large V, small E** — adjacency list is most memory-efficient here.

## 15. Variations

| Variation                      | What Changes                                        | When Used                          | Importance        |
|--------------------------------|-----------------------------------------------------|------------------------------------|-------------------|
| <b>Vector of unordered_set</b> | <code>vector&lt;unordered_set&lt;int&gt;&gt;</code> | O(1) edge lookup, more memory      | Medium            |
| <b>Vector of list</b>          | <code>vector&lt;list&lt;int&gt;&gt;</code>          | O(1) edge deletion from middle     | Low               |
| <b>Edge list</b>               | <code>vector&lt;tuple&lt;int,int,int&gt;&gt;</code> | Kruskal's MST, sorting edges       | High              |
| <b>CSR format</b>              | Compressed Sparse Row                               | Very large graphs, memory critical | Low (specialized) |

## 16. Related Concepts

- **Adjacency Matrix** — Alternative for dense graphs, O(1) edge lookup,  $O(V^2)$  space.
- **Edge List** — Simple list of edges; used in Kruskal's, Bellman-Ford.
- **BFS/DFS** — Traversal algorithms that work naturally with adjacency lists.
- **DSU** — Can replace adjacency list when only connectivity matters.

## 17. Practice Problems

### Easy

- **Find Center of Star Graph** — LeetCode (1791) — Adjacency list degree check
- **Find if Path Exists in Graph** — LeetCode (1971) — BFS/DFS on adjacency list

### Medium

- **Clone Graph** — LeetCode (133) — DFS with map on adjacency list
- **Course Schedule** — LeetCode (207) — Cycle detection using adjacency list
- **Number of Connected Components** — LeetCode (323) — DFS on adjacency list

### Hard

- **Alien Dictionary** — LeetCode (269) — Topological sort on adjacency list
- **Minimum Number of Vertices to Reach All Nodes** — LeetCode (1557) — In-degree from adjacency list
- **Reconstruct Itinerary** — LeetCode (332) — Hierholzer's algorithm on adjacency list

## 18. Interview Explanation

"An adjacency list is my default graph representation. I use a vector of vectors where `adj[u]` stores all neighbours of vertex `u`. It takes  $O(V+E)$  space, which is optimal for sparse graphs. Adding an edge is  $O(1)$ . Iterating over neighbours is  $O(\text{degree}(u))$ , which is optimal. Checking edge existence is  $O(\text{degree}(u))$  in the worst case — if I need faster lookups, I use `unordered_set` per vertex. This representation works naturally with BFS, DFS, Dijkstra, and most graph algorithms."

## 19. Revision Notes

- Default graph representation: `vector<vector<int>> adj(V)`.
- Add undirected: `adj[u].push_back(v); adj[v].push_back(u)`.
- Add directed: `adj[u].push_back(v)` only.
- Weighted: `vector<vector<pair<int,int>>> adj`.
- O(1) add, O(degree) check, O(V+E) space.
- Use `unordered_set` for O(1) edge lookup.
- Use edge list (`vector<tuple<int,int,int>>`) for Kruskal's.

## 20. Final Cheat Sheet

| Aspect      | Summary                                                                   |
|-------------|---------------------------------------------------------------------------|
| When to use | Default for all graph problems (sparse or moderate density)               |
| Structure   | <code>vector&lt;vector&lt;int&gt;&gt; adj(V)</code> — unweighted          |
| Weighted    | <code>vector&lt;vector&lt;pair&lt;int,int&gt;&gt;&gt; adj(V)</code>       |
| Add edge    | O(1) — <code>adj[u].push_back(v)</code>                                   |
| Check edge  | O(degree) — scan list, or use <code>unordered_set</code>                  |
| Space       | O(V + E) — optimal for sparse graphs                                      |
| Key traps   | Forgetting reverse edge for undirected; passing by value; duplicate edges |
| Must know   | BFS, DFS, Dijkstra, topological sort on adjacency list                    |

## 7. Degree

### 1. Overview

**Degree** of a vertex in a graph is the number of edges incident to it. It is a fundamental property that appears in many graph algorithms and problems.

- **Undirected graph:** Degree = number of neighbours.
- **Directed graph:** In-degree = incoming edges, Out-degree = outgoing edges.
- **Sum of degrees rule:** Sum of degrees =  $2 \times E$  (for undirected graphs).

### 2. Intuition

Think of **popularity in a social network**. If you have 500 friends, your degree is 500. In a directed graph (Twitter), your out-degree is how many people you follow, and your in-degree is how many followers you have.

**Step-by-step reasoning:**

- In an undirected graph, each edge contributes 1 to the degree of both endpoints.
- Therefore, sum of degrees =  $2 \times E$ .
- In a directed graph, each edge contributes 1 to out-degree of source and 1 to in-degree of destination.
- A vertex with degree 0 is isolated.

- Degree distributions tell us about the graph structure (e.g., power-law graphs).

### 3. When to Use It

- For **handshaking lemma** problems (sum of degrees =  $2 \times E$ ).
- To find the **center of a star graph** (one vertex has degree  $V-1$ , others have degree 1).
- For **topological sorting** (Kahn's algorithm uses in-degree).
- To find **source/sink nodes** in directed graphs.
- For **graph theory proofs** and validations.
- To detect **isolated vertices**.
- For **Eulerian path/circuit** problems (checking degree parity).

**Trigger phrases:** "degree", "popularity", "in-degree", "out-degree", "handshaking", "isolated", "source", "sink", "Eulerian", "star graph", "followers".

### 4. When Not to Use It

- When degree alone doesn't capture the structure you need (e.g., degree doesn't tell you about connectivity patterns).
- Don't confuse degree with path length or distance.
- In weighted graphs, degree counts edges, not total weight — use "weighted degree" or "strength" if you need sum of weights.

### 5. Core Concepts

| Concept                      | Explanation                       | Why It Matters                                |
|------------------------------|-----------------------------------|-----------------------------------------------|
| <b>Degree (undirected)</b>   | <code>adj[u].size()</code>        | Number of neighbours                          |
| <b>In-degree (directed)</b>  | Number of edges entering a vertex | Topological sort, followers                   |
| <b>Out-degree (directed)</b> | Number of edges leaving a vertex  | Following, influence spread                   |
| <b>Handshaking Lemma</b>     | Sum of degrees = $2 \times E$     | Validates graph data, used in proofs          |
| <b>Degree sequence</b>       | List of degrees of all vertices   | Graph isomorphism check (necessary condition) |
| <b>Isolated vertex</b>       | Degree = 0                        | Disconnected component                        |
| <b>Leaf</b>                  | Degree = 1 (in a tree)            | Tree properties, DFS leaves                   |
| <b>Regular graph</b>         | All vertices have the same degree | Special graph class                           |

### 6. Step-by-Step Algorithm (Computing Degrees)

**Undirected graph:**

1. For each vertex  $u$ : `degree[u] = adj[u].size()`.

2. Sum of degrees =  $\sum \text{degree}[u] = 2 \times E$ .

### Directed graph:

1. For each vertex  $u$ : `outDegree[u] = adj[u].size()`.
2. Initialize `inDegree[u] = 0` for all  $u$ .
3. For each edge  $(u, v)$ : `inDegree[v]++`.
4. Sum of out-degrees = Sum of in-degrees =  $E$ .

## 7. Dry Run

**Undirected graph:**  $V = 4$ , edges = [(0,1), (0,2), (1,2), (2,3)]

| Vertex | adj list  | Degree |
|--------|-----------|--------|
| 0      | [1, 2]    | 2      |
| 1      | [0, 2]    | 2      |
| 2      | [0, 1, 3] | 3      |
| 3      | [2]       | 1      |

Sum of degrees =  $2 + 2 + 3 + 1 = 8 = 2 \times 4 = 2E \checkmark$

**Directed graph:**  $V = 4$ , edges = [(0→1), (0→2), (1→2), (2→3), (3→1)]

| Vertex | adj list | Out-degree | In-degree     |
|--------|----------|------------|---------------|
| 0      | [1, 2]   | 2          | 0             |
| 1      | [2]      | 1          | 2 (from 0, 3) |
| 2      | [3]      | 1          | 2 (from 0, 1) |
| 3      | [1]      | 1          | 1 (from 2)    |

Sum out =  $2+1+1+1 = 5 = E \checkmark$ , Sum in =  $0+2+2+1 = 5 = E \checkmark$

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DegreeAnalyzer {
private:
    int V;
    vector<vector<int>> adj;

public:
    DegreeAnalyzer(int vertices) : V(vertices) {
        adj.resize(V);
    }
}
```

```

void addUndirectedEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void addDirectedEdge(int u, int v) {
    adj[u].push_back(v);
}

// Undirected degree
vector<int> undirectedDegrees() {
    vector<int> deg(V);
    for (int u = 0; u < V; u++)
        deg[u] = adj[u].size();
    return deg;
}

// Directed in and out degrees
pair<vector<int>, vector<int>> directedDegrees() {
    vector<int> inDeg(V, 0), outDeg(V);
    for (int u = 0; u < V; u++) {
        outDeg[u] = adj[u].size();
        for (int v : adj[u])
            inDeg[v]++;
    }
    return {inDeg, outDeg};
}

// Handshaking lemma check
bool checkHandshaking() {
    long long sum = 0;
    for (int u = 0; u < V; u++)
        sum += adj[u].size();
    return sum % 2 == 0 && sum / 2 == (long long)(
        // We'd need E count separately for exact check
        sum / 2
    );
}

// Find vertices with degree 0
vector<int> isolatedVertices() {
    vector<int> isolated;
    for (int u = 0; u < V; u++)
        if (adj[u].empty())
            isolated.push_back(u);
    return isolated;
}

// Find sources (in-degree = 0) in directed graph
vector<int> sources() {
    auto [inDeg, _] = directedDegrees();
    vector<int> src;
    for (int u = 0; u < V; u++)

```

```

        if (inDeg[u] == 0)
            src.push_back(u);
        return src;
    }

    // Find sinks (out-degree = 0) in directed graph
    vector<int> sinks() {
        vector<int> snk;
        for (int u = 0; u < V; u++)
            if (adj[u].empty())
                snk.push_back(u);
        return snk;
    }

    // Degree sequence (sorted)
    vector<int> degreeSequence() {
        vector<int> deg = undirectedDegrees();
        sort(deg.begin(), deg.end());
        return deg;
    }
};

int main() {
    DegreeAnalyzer g(4);
    g.addDirectedEdge(0, 1);
    g.addDirectedEdge(0, 2);
    g.addDirectedEdge(1, 2);
    g.addDirectedEdge(2, 3);
    g.addDirectedEdge(3, 1);

    auto [inDeg, outDeg] = g.directedDegrees();
    for (int i = 0; i < 4; i++)
        cout << "Vertex " << i << ": in=" << inDeg[i]
              << " out=" << outDeg[i] << "\n";

    auto src = g.sources();
    cout << "Sources: ";
    for (int s : src) cout << s << " ";
    cout << "\n";

    auto snk = g.sinks();
    cout << "Sinks: ";
    for (int s : snk) cout << s << " ";
    cout << "\n";
    return 0;
}

```

## 9. Python Implementation

```

from typing import List, Tuple

```

```

class DegreeAnalyzer:
    def __init__(self, vertices: int):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_undirected_edge(self, u: int, v: int):
        self.adj[u].append(v)
        self.adj[v].append(u)

    def add_directed_edge(self, u: int, v: int):
        self.adj[u].append(v)

    def undirected_degrees(self) -> List[int]:
        return [len(self.adj[u]) for u in range(self.V)]

    def directed_degrees(self) -> Tuple[List[int], List[int]]:
        in_deg = [0] * self.V
        out_deg = [0] * self.V
        for u in range(self.V):
            out_deg[u] = len(self.adj[u])
            for v in self.adj[u]:
                in_deg[v] += 1
        return in_deg, out_deg

    def isolated_vertices(self) -> List[int]:
        return [u for u in range(self.V) if not self.adj[u]]

    def sources(self) -> List[int]:
        in_deg, _ = self.directed_degrees()
        return [u for u in range(self.V) if in_deg[u] == 0]

    def sinks(self) -> List[int]:
        return [u for u in range(self.V) if not self.adj[u]]

    def degree_sequence(self) -> List[int]:
        return sorted(self.undirected_degrees())

# Example
g = DegreeAnalyzer(4)
g.add_directed_edge(0, 1)
g.add_directed_edge(0, 2)
g.add_directed_edge(1, 2)
g.add_directed_edge(2, 3)
g.add_directed_edge(3, 1)

in_deg, out_deg = g.directed_degrees()
for i in range(4):
    print(f"Vertex {i}: in={in_deg[i]} out={out_deg[i]}")
print(f"Sources: {g.sources()}")
print(f"Sinks: {g.sinks()}")

```

## 10. Code Explanation

- **undirectedDegrees()**: Simply returns `adj[u].size()` for each vertex. Each edge is counted twice (once for each endpoint), so the sum is  $2E$ .
- **directedDegrees()**: `outDeg[u] = adj[u].size()` directly. `inDeg[v]` is incremented while scanning all edges. This is  $O(V+E)$ .
- **sources()**: Vertices with in-degree 0. These are starting points in topological sort.
- **sinks()**: Vertices with out-degree 0. These are end points in dependency chains.
- **isolatedVertices()**: Vertices with degree 0 — no edges at all.
- **degreeSequence()**: Sorted list of degrees. Used for checking if a graph is a tree (degree sequence of tree has specific properties).

## 11. Complexity Analysis

| Operation              | Complexity               |
|------------------------|--------------------------|
| Degree (undirected)    | $O(V)$ to compute all    |
| In-degree (directed)   | $O(V+E)$ to compute all  |
| Out-degree (directed)  | $O(V)$ to compute all    |
| Find sources/sinks     | $O(V+E)$                 |
| Find isolated vertices | $O(V)$                   |
| Space                  | $O(V)$ for degree arrays |

## 12. Common Patterns

| Pattern                  | Identification                           | Approach                                          | Problems                      |
|--------------------------|------------------------------------------|---------------------------------------------------|-------------------------------|
| <b>Kahn's Algorithm</b>  | Topological sort                         | Use in-degree array, process zero in-degree nodes | Course Schedule               |
| <b>Handshaking Lemma</b> | "Number of handshakes", "sum of degrees" | $E = (\text{sum of degrees}) / 2$                 | Count pairs, graph validation |
| <b>Star Graph Center</b> | "One node connected to all others"       | Find vertex with degree = $V-1$                   | Find Center of Star Graph     |
| <b>Eulerian Path</b>     | "Traverse every edge exactly once"       | Check degree parity conditions                    | Eulerian Path/Circuit         |
| <b>Graph Validation</b>  | "Is this a valid graph?"                 | Sum of degrees must be even                       | Degree sequence validation    |

## 13. Common Mistakes

- Forgetting that in an undirected graph, each edge is incident to 2 vertices, so sum of degrees =  $2E$ .
- Using `adj[u].size()` for in-degree in a directed graph (it gives out-degree, not in-degree).
- Not updating in-degree when adding edges dynamically.

- Assuming degree = 0 means the vertex doesn't exist (it might just be isolated).
- Confusing degree with weighted degree (sum of weights of incident edges).

## 14. Edge Cases

- **Single vertex, no edges** — degree = 0.
- **Complete graph** — each vertex has degree =  $V-1$ .
- **Graph with self-loops** — self-loop contributes 2 to degree (in undirected) or 1 to out-degree and 1 to in-degree (in directed).
- **Empty graph** — all degrees = 0.
- **Regular graph** — all vertices have the same degree.
- **Bipartite graph** — degree distribution can vary.

## 15. Variations

| Variation                         | What Changes                      | When Used                           | Importance |
|-----------------------------------|-----------------------------------|-------------------------------------|------------|
| <b>Weighted degree (strength)</b> | Sum of incident edge weights      | Weighted network analysis           | Medium     |
| <b>Normalized degree</b>          | Degree / $(V-1)$                  | Comparing graphs of different sizes | Low        |
| <b>Degree centrality</b>          | Degree as a measure of importance | Social network analysis             | Medium     |
| <b>PageRank</b>                   | Weighted variant of in-degree     | Web search ranking                  | High       |

## 16. Related Concepts

- **Adjacency List** — Degree is directly `adj[u].size()`.
- **Topological Sort** — Uses in-degree for Kahn's algorithm.
- **Eulerian Path** — Degree parity determines existence.
- **Handshaking Lemma** — Fundamental graph theory result.

## 17. Practice Problems

### Easy

- **Find Center of Star Graph** — LeetCode (1791) — Degree counting
- **Find the Town Judge** — LeetCode (997) — In-degree/out-degree comparison

### Medium

- **Minimum Number of Vertices to Reach All Nodes** — LeetCode (1557) — In-degree zero sources
- **Course Schedule** — LeetCode (207) — In-degree for topological sort

### Hard

- **Eulerian Circuit** — LeetCode (332) Reconstruct Itinerary — Degree conditions

- **Valid Arrangement of Pairs** — LeetCode (2097) — Eulerian path using degree

## 18. Interview Explanation

"Degree is the number of edges incident to a vertex. In undirected graphs, it's simply the size of the adjacency list. In directed graphs, we have in-degree (incoming edges) and out-degree (outgoing edges). The handshaking lemma states that the sum of degrees in an undirected graph is twice the number of edges. I use degree in topological sort (Kahn's algorithm processes nodes with in-degree zero), to find sources and sinks, and to check Eulerian path conditions."

## 19. Revision Notes

- Undirected: `deg[u] = adj[u].size()`, `sum = 2E`.
- Directed: `outDeg[u] = adj[u].size()`, `inDeg[v]++` for each edge (u,v).
- Sources = in-degree 0, sinks = out-degree 0.
- Handshaking lemma: sum of degrees is always even.
- Leaf in tree: degree = 1.
- Center of star: degree = V-1.

## 20. Final Cheat Sheet

| Aspect              | Summary                                                                |
|---------------------|------------------------------------------------------------------------|
| When to use         | Graph analysis, topological sort, Eulerian path, source/sink detection |
| Undirected degree   | <code>adj[u].size()</code> — $O(1)$                                    |
| Directed in-degree  | Scan edges or maintain array — $O(V+E)$                                |
| Directed out-degree | <code>adj[u].size()</code> — $O(1)$                                    |
| Key formula         | Sum of degrees = $2E$ (undirected)                                     |
| Key traps           | Don't confuse in-degree and out-degree in directed graphs              |
| Must know           | Handshaking lemma, Kahn's algorithm, Eulerian path conditions          |

# 8. Connected Components

## 1. Overview

A **connected component** is a maximal set of vertices in an undirected graph where every pair of vertices has a path between them. If the graph is directed, we have **weakly connected components** (ignoring direction) and **strongly connected components** (mutual reachability respecting direction).

## 2. Intuition

Think of **islands in an archipelago**. Each island is a connected component. You can walk from any point on one island to any other point on the same island, but you cannot walk to a different island without a boat.

Another analogy: **Friend groups at a party**. People who know each other (directly or indirectly) form a group. Two people from different groups don't know each other.

**Step-by-step reasoning:**

- Start from any vertex and traverse (BFS/DFS) to find all reachable vertices.
- Those reachable vertices form one connected component.
- Pick any unvisited vertex and repeat.
- The number of components = number of times we start a new traversal.
- For directed graphs, SCCs are more complex (Kosaraju's / Tarjan's algorithm).

3. When to Use It

- **Counting provinces/groups** ("number of friend circles", "number of provinces").
- **Checking if a graph is connected** (single component).
- **Graph partitioning** problems.
- **Adding edges to connect the graph** ("minimum edges to make graph connected").
- **DSU** (Disjoint Set Union) for dynamic connectivity.
- **Image segmentation** (connected components in grid / 4-directional).

**Trigger phrases:** "connected components", "provinces", "friend circles", "groups", "islands", "clusters", "partition", "connected or not", "make graph connected".

4. When Not to Use It

- When the graph is directed and you need **strongly connected components** — use Kosaraju's or Tarjan's instead.
- When you need **dynamic connectivity** (edges added/removed over time) — use DSU with rollbacks or Link-Cut Tree.
- When you just need to check if two specific vertices are connected — BFS/DFS from source is enough (no need to find all components).

5. Core Concepts

| Concept                    | Explanation                                            | Why It Matters                                         |
|----------------------------|--------------------------------------------------------|--------------------------------------------------------|
| <b>Connected component</b> | Maximal set of vertices with mutual paths              | Building block of graph analysis                       |
| <b>Weakly connected</b>    | Directed graph, ignoring direction                     | Simpler connectivity notion for directed graphs        |
| <b>Strongly connected</b>  | Directed graph, respecting direction                   | Every node can reach every other node in the component |
| <b>Component ID</b>        | Label assigned to each vertex indicating its component | Used to group vertices, compare components             |
| <b>Component size</b>      | Number of vertices in a component                      | Find largest/smallest component                        |

| Concept | Explanation                       | Why It Matters                                      |
|---------|-----------------------------------|-----------------------------------------------------|
| DSU     | Disjoint Set Union data structure | Efficiently tracks components during edge additions |

## 6. Step-by-Step Algorithm (Finding Connected Components)

### Using DFS/BFS (static graph):

1. Initialize `visited[V] = {false}`.
2. Initialize `components = 0`.
3. For each vertex `u` from 0 to `V-1`:
  - If not `visited[u]`:
    - Increment `components`.
    - Start DFS/BFS from `u`, marking all reachable vertices as visited.
4. Return `components`.

### Using DSU (dynamic graph):

1. Initialize DSU with `V` elements.
2. For each edge `(u, v)`: `union(u, v)`.
3. Count distinct roots = number of components.

## 7. Dry Run

**Input:** `V = 7`, undirected edges = `[(0,1), (1,2), (3,4), (5,6)]`

| Step | Start at                       | Visited Set                  | Components |
|------|--------------------------------|------------------------------|------------|
| 1    | <code>u=0</code>               | <code>{}</code>              | 0          |
| 2    | DFS from 0                     | <code>{0, 1, 2}</code>       | 1          |
| 3    | <code>u=3</code> (not visited) | <code>{0,1,2,3,4}</code>     | 2          |
| 4    | <code>u=5</code> (not visited) | <code>{0,1,2,3,4,5,6}</code> | 3          |
| 5    | <code>u=6</code> (visited)     | all visited                  | 3          |

**Components:** `{0,1,2}`, `{3,4}`, `{5,6}` — 3 components.

**Component sizes:** 3, 2, 2.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class ConnectedComponents {
private:
    int V;
    vector<vector<int>> adj;
```

```
public:
    ConnectedComponents(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // DFS to mark all vertices in a component
    void dfs(int u, vector<bool> &visited, vector<int> &component) {
        visited[u] = true;
        component.push_back(u);
        for (int v : adj[u])
            if (!visited[v])
                dfs(v, visited, component);
    }

    // Find all connected components
    vector<vector<int>> findComponents() {
        vector<bool> visited(V, false);
        vector<vector<int>> components;

        for (int u = 0; u < V; u++) {
            if (!visited[u]) {
                vector<int> comp;
                dfs(u, visited, comp);
                components.push_back(comp);
            }
        }
        return components;
    }

    // Count components
    int countComponents() {
        vector<bool> visited(V, false);
        int count = 0;

        for (int u = 0; u < V; u++) {
            if (!visited[u]) {
                count++;
                // BFS to mark all reachable vertices
                queue<int> q;
                q.push(u);
                visited[u] = true;
                while (!q.empty()) {
                    int cur = q.front(); q.pop();
                    for (int v : adj[cur])
                        if (!visited[v]) {
                            visited[v] = true;
                            q.push(v);
                        }
                }
            }
        }
    }
}
```

```

        }
    }
    }
    return count;
}

// Check if graph is connected
bool isConnected() {
    return countComponents() == 1;
}

// Get component IDs for each vertex
vector<int> getComponentIds() {
    vector<int> compId(V, -1);
    int id = 0;
    for (int u = 0; u < V; u++) {
        if (compId[u] == -1) {
            queue<int> q;
            q.push(u);
            compId[u] = id;
            while (!q.empty()) {
                int cur = q.front(); q.pop();
                for (int v : adj[cur])
                    if (compId[v] == -1) {
                        compId[v] = id;
                        q.push(v);
                    }
            }
            id++;
        }
    }
    return compId;
}

// Minimum edges to make graph connected
int minEdgesToConnect() {
    int comps = countComponents();
    return comps - 1; // need comps-1 edges to connect all components
}

};

// DSU-based connected components (dynamic)
class DSU {
private:
    vector<int> parent, rank;
    int components;

public:
    DSU(int n) : components(n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

```

```

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void unite(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return;
    if (rank[px] < rank[py]) swap(px, py);
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    components--;
}

int countComponents() { return components; }
};

int main() {
    ConnectedComponents g(7);
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(3, 4);
    g.addEdge(5, 6);

    cout << "Number of components: " << g.countComponents() << "\n"; // 3
    cout << "Is connected: " << g.isConnected() << "\n"; // 0 (false)
    cout << "Min edges to connect: " << g.minEdgesToConnect() << "\n"; // 2

    auto comps = g.findComponents();
    for (int i = 0; i < comps.size(); i++) {
        cout << "Component " << i << ": ";
        for (int v : comps[i]) cout << v << " ";
        cout << "\n";
    }

    // DSU approach
    DSU dsu(7);
    dsu.unite(0, 1); dsu.unite(1, 2);
    dsu.unite(3, 4);
    dsu.unite(5, 6);
    cout << "DSU components: " << dsu.countComponents() << "\n"; // 3
    return 0;
}

```

## 9. Python Implementation

```

from collections import deque
from typing import List

class ConnectedComponents:

```

```

def __init__(self, vertices: int):
    self.V = vertices
    self.adj = [[] for _ in range(vertices)]

def add_edge(self, u: int, v: int):
    self.adj[u].append(v)
    self.adj[v].append(u)

def dfs(self, u: int, visited: List[bool], component: List[int]):
    visited[u] = True
    component.append(u)
    for v in self.adj[u]:
        if not visited[v]:
            self.dfs(v, visited, component)

def find_components(self) -> List[List[int]]:
    visited = [False] * self.V
    components = []
    for u in range(self.V):
        if not visited[u]:
            comp = []
            self.dfs(u, visited, comp)
            components.append(comp)
    return components

def count_components(self) -> int:
    visited = [False] * self.V
    count = 0
    for u in range(self.V):
        if not visited[u]:
            count += 1
            # BFS
            q = deque([u])
            visited[u] = True
            while q:
                cur = q.popleft()
                for v in self.adj[cur]:
                    if not visited[v]:
                        visited[v] = True
                        q.append(v)
    return count

def is_connected(self) -> bool:
    return self.count_components() == 1

def min_edges_to_connect(self) -> int:
    return self.count_components() - 1

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

```

```

def find(self, x: int) -> int:
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

def unite(self, x: int, y: int):
    px, py = self.find(x), self.find(y)
    if px == py:
        return
    if self.rank[px] < self.rank[py]:
        px, py = py, px
    self.parent[py] = px
    if self.rank[px] == self.rank[py]:
        self.rank[px] += 1
    self.components -= 1

def count_components(self) -> int:
    return self.components

# Example
g = ConnectedComponents(7)
g.add_edge(0, 1)
g.add_edge(1, 2)
g.add_edge(3, 4)
g.add_edge(5, 6)
print(f"Components: {g.count_components()}") # 3
print(f"Connected: {g.is_connected()}") # False
print(f"Min edges: {g.min_edges_to_connect()}") # 2

```

## 10. Code Explanation

- **dfs(u, visited, component)**: Recursive DFS that adds visited vertices to the current component list.
- **findComponents()**: Iterates over all vertices. For each unvisited vertex, starts a DFS and collects all vertices in that component. Returns a list of components.
- **countComponents()**: Same logic but only counts, doesn't store components. Uses BFS for marking.
- **isConnected()**: Returns true if there's exactly 1 component.
- **minEdgesToConnect()**: To connect  $k$  components, you need  $k-1$  edges. This is a common interview question.
- **getComponentIds()**: Assigns a component ID (0, 1, 2, ...) to each vertex for quick lookup.
- **DSU class**: The **find** uses path compression, **unite** uses union by rank. **components** decrements on each successful union.

## 11. Complexity Analysis

| Operation                     | Complexity |
|-------------------------------|------------|
| Find all components (DFS/BFS) | $O(V + E)$ |
| Count components              | $O(V + E)$ |

| Operation                  | Complexity                   |
|----------------------------|------------------------------|
| Check if connected         | $O(V + E)$                   |
| Min edges to connect       | $O(V + E)$                   |
| DSU operations (amortized) | $O(\alpha(V))$ per operation |
| Space                      | $O(V + E)$                   |

## 12. Common Patterns

| Pattern                     | Identification                             | Approach                                  | Problems                                       |
|-----------------------------|--------------------------------------------|-------------------------------------------|------------------------------------------------|
| <b>Count Components</b>     | "Number of provinces",<br>"friend circles" | DFS/BFS from each unvisited vertex        | Number of Provinces                            |
| <b>Make Graph Connected</b> | "Minimum edges to connect"                 | Count components, answer = components - 1 | Number of Operations to Make Network Connected |
| <b>Largest Component</b>    | "Biggest group",<br>"largest island"       | Track component sizes during DFS          | Largest Component Size by Common Factor        |
| <b>Component ID Lookup</b>  | "Are u and v in same component?"           | DSU or component ID array                 | Redundant Connection                           |
| <b>Dynamic Connectivity</b> | "Add edges and check connectivity"         | DSU with union operations                 | Graph Valid Tree, Connecting Cities            |

## 13. Common Mistakes

- Forgetting to handle **disconnected components** in graph traversal — DFS/BFS from a single source only visits one component.
- Using **visited** array but not resetting it between test cases.
- Forgetting that an undirected graph with  $V$  vertices and  $E$  edges may have multiple components.
- In DSU, not using path compression — leads to  $O(V)$  per find.
- In DSU, not using union by rank/size — leads to a skewed tree.
- Assuming **minEdgesToConnect()** = components - 1 always works — it does for undirected graphs, but for directed graphs, you need more edges.

## 14. Edge Cases

- **Empty graph** ( $V = 0$ ) — 0 components.
- **Single vertex, no edges** — 1 component of size 1.
- **Complete graph** — 1 component.
- **All vertices isolated** —  $V$  components, each of size 1.
- **Graph with one vertex connected to all others** — 1 component.
- **Tree** — 1 component,  $V-1$  edges.

## 15. Variations

| Variation                            | What Changes                           | When Used                       | Importance |
|--------------------------------------|----------------------------------------|---------------------------------|------------|
| <b>Strongly Connected Components</b> | Directed graph, mutual reachability    | Kosaraju's / Tarjan's algorithm | Very High  |
| <b>2-Edge-Connected Components</b>   | Components without bridges             | Network reliability             | Medium     |
| <b>Biconnected Components</b>        | Components without articulation points | Graph vulnerability analysis    | Medium     |
| <b>Connected Components in Grid</b>  | 4/8-directional connectivity           | Image processing, maze problems | Very High  |

## 16. Related Concepts

- **DFS/BFS** — The traversal algorithms used to find components.
- **DSU** — Data structure for dynamic connectivity (edge additions).
- **Strongly Connected Components** — Directed graph version.
- **Minimum Spanning Tree** — Connects all vertices with minimum total weight.

## 17. Practice Problems

### Easy

- **Number of Provinces** — LeetCode (547) — Connected components
- **Find if Path Exists in Graph** — LeetCode (1971) — Component membership

### Medium

- **Number of Operations to Make Network Connected** — LeetCode (1319) — Components - 1
- **Regions Cut By Slashes** — LeetCode (959) — Connected components in grid
- **Graph Valid Tree** — LeetCode (261) — Single component +  $V-1$  edges

### Hard

- **Largest Component Size by Common Factor** — LeetCode (952) — DSU with factor-based edges
- **Number of Ways to Arrive at Destination** — LeetCode (1976) — Dijkstra + component counting
- **Minimum Cost to Connect Two Groups of Points** — LeetCode (1595) — DP + DSU

## 18. Interview Explanation

"Connected components are maximal sets of vertices where every pair has a path between them. I find them by running DFS/BFS from each unvisited vertex. Each traversal discovers one component. The time complexity is  $O(V+E)$ . For dynamic connectivity, I use DSU with path compression and union by rank. To connect  $k$  components, I need  $k-1$  edges. For directed graphs, I use Kosaraju's or Tarjan's algorithm for strongly connected components."

## 19. Revision Notes

- Run DFS/BFS from each unvisited vertex → each traversal = 1 component.

- `components = 1` means graph is connected.
- `minEdges = components - 1` to connect all components.
- DSU: `find` with path compression, `unite` with union by rank.
- DSU `components` decrements on each successful union.
- Grid components: 4-directional or 8-directional connectivity.
- For directed graphs, use SCC algorithms (Kosaraju's, Tarjan's).

## 20. Final Cheat Sheet

| Aspect                      | Summary                                                          |
|-----------------------------|------------------------------------------------------------------|
| <b>When to use</b>          | Counting groups, checking connectivity, graph partitioning       |
| <b>Algorithm</b>            | DFS/BFS from each unvisited vertex                               |
| <b>Complexity</b>           | $O(V + E)$ for static graph                                      |
| <b>DSU</b>                  | $O(\alpha(V))$ per operation for dynamic graph                   |
| <b>Min edges to connect</b> | <code>components - 1</code>                                      |
| <b>Key traps</b>            | Handle disconnected components, reset visited between test cases |
| <b>Must know</b>            | DFS/BFS component counting, DSU, min edges to connect            |

## 9. Graph Traversal

### 1. Overview

**Graph traversal** refers to the process of visiting all vertices in a graph systematically. The two fundamental traversal algorithms are **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.

- **BFS:** Explores neighbours first (level by level). Uses a queue.
- **DFS:** Explores as deep as possible along each branch before backtracking. Uses a stack (or recursion).

### 2. Intuition

**BFS analogy: Ripples in a pond.** When you drop a stone, ripples spread outward in all directions, one layer at a time. BFS visits nodes at distance 0, then distance 1, then distance 2, etc.

**DFS analogy: Exploring a maze.** You go down one path until you hit a dead end, then backtrack and try the next path. DFS goes deep first, then wide.

#### Step-by-step reasoning (BFS):

1. Start at the source node.
2. Visit all nodes at distance 1 from source.
3. Then all nodes at distance 2.
4. Continue until all reachable nodes are visited.
5. BFS guarantees shortest path in unweighted graphs.

**Step-by-step reasoning (DFS):**

1. Start at the source node.
2. Go to the first unvisited neighbour.
3. Recurse (keep going deeper).
4. When no unvisited neighbours remain, backtrack.
5. DFS is used for connectivity, cycle detection, topological sorting.

### 3. When to Use It

**BFS:**

- Shortest path in unweighted graphs.
- Level-order traversal.
- Multi-source problems (e.g., nearest distance from a set of sources).
- Finding connected components.
- Web crawling (breadth-first).

**DFS:**

- Exploring all paths or configurations.
- Cycle detection.
- Topological sorting.
- Connected components.
- Solving puzzles (maze, Sudoku) with backtracking.
- Tree traversals (preorder, inorder, postorder).

**Trigger phrases:** "traverse", "visit all nodes", "shortest path", "explore", "backtracking", "level order", "depth", "topological order", "cycle detection".

### 4. When Not to Use It

- BFS for shortest path in **weighted graphs** — use Dijkstra's.
- DFS for shortest path — DFS doesn't guarantee shortest path; BFS does.
- BFS when memory is tight in a very deep graph — BFS queue can grow large.
- DFS when the recursion depth could exceed the stack limit ( $V > 10^5$ ) — use iterative DFS.
- Both BFS and DFS are  $O(V+E)$  — don't use them if the problem only needs DSU for connectivity.

### 5. Core Concepts

| Concept              | Explanation                                                            | Why It Matters                   |
|----------------------|------------------------------------------------------------------------|----------------------------------|
| <b>Visited array</b> | <code>bool visited[V]</code> — tracks which vertices have been visited | Prevents infinite loops          |
| <b>Queue (BFS)</b>   | FIFO data structure                                                    | Ensures level-by-level traversal |
| <b>Stack (DFS)</b>   | LIFO data structure (or recursion call stack)                          | Ensures deep-first traversal     |
| <b>Parent array</b>  | Tracks which vertex led to current vertex                              | Reconstruct paths                |

| Concept                      | Explanation                                         | Why It Matters                     |
|------------------------------|-----------------------------------------------------|------------------------------------|
| <b>Distance array</b>        | Stores distance from source                         | Shortest path in BFS               |
| <b>Recursion stack (DFS)</b> | Stack of vertices currently on the DFS path         | Cycle detection in directed graphs |
| <b>Discovery/Finish time</b> | Time when vertex is first visited / fully processed | Used in topological sort, SCC      |

## 6. Step-by-Step Algorithm

### BFS (Breadth-First Search)

1. Create `visited[V] = {false}`, `dist[V] = {INF}`, `parent[V] = {-1}`.
2. Create queue `q`.
3. Mark `src` as visited, set `dist[src] = 0`, push `src` to `q`.
4. While `q` is not empty:
  - Pop `u` from front of `q`.
  - For each neighbour `v` of `u`:
    - If not visited:
      - Mark visited, set `dist[v] = dist[u] + 1`, set `parent[v] = u`, push `v` to `q`.
5. BFS complete.

### DFS (Depth-First Search)

1. Create `visited[V] = {false}`.
2. Call `dfs(src)`:
  - Mark `u` as visited.
  - For each neighbour `v` of `u`:
    - If not visited: call `dfs(v)`.
  - (For directed cycles: also track `recStack[]`).

## 7. Dry Run

**Graph:**  $V = 6$ , undirected edges = [(0,1), (0,2), (1,3), (1,4), (2,4), (3,5), (4,5)]

BFS from 0:

| Step | Queue                   | Visited       | dist               |
|------|-------------------------|---------------|--------------------|
| 0    | [0]                     | {0}           | [0, ∞, ∞, ∞, ∞, ∞] |
| 1    | Pop 0, push 1,2         | {0,1,2}       | [0, 1, 1, ∞, ∞, ∞] |
| 2    | Pop 1, push 3,4         | {0,1,2,3,4}   | [0, 1, 1, 2, 2, ∞] |
| 3    | Pop 2, skip (4 visited) | {0,1,2,3,4}   | [0, 1, 1, 2, 2, ∞] |
| 4    | Pop 3, push 5           | {0,1,2,3,4,5} | [0, 1, 1, 2, 2, 3] |
| 5    | Pop 4, pop 5 (no new)   | all visited   | [0, 1, 1, 2, 2, 3] |

**BFS distances from 0:**  $0 \rightarrow 0 = 0$ ,  $0 \rightarrow 1 = 1$ ,  $0 \rightarrow 2 = 1$ ,  $0 \rightarrow 3 = 2$ ,  $0 \rightarrow 4 = 2$ ,  $0 \rightarrow 5 = 3$

DFS from 0:

| Step | Stack                 | Visited     | Current Path |
|------|-----------------------|-------------|--------------|
| 0    | [0]                   | {0}         | 0            |
| 1    | [0,1]                 | {0,1}       | 0→1          |
| 2    | [0,1,3]               | {0,1,3}     | 0→1→3        |
| 3    | [0,1,3,5]             | {0,1,3,5}   | 0→1→3→5      |
| 4    | [0,1,3] (backtrack 5) | all visited | 0→1→3        |
| 5    | [0,1] (backtrack 3)   | all visited | 0→1          |
| 6    | [0,1,4]               | all visited | 0→1→4        |
| 7    | [0] (backtrack)       | all visited | 0            |
| 8    | [0,2]                 | all visited | 0→2          |

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class GraphTraversal {
private:
    int V;
    vector<vector<int>> adj;

public:
    GraphTraversal(int vertices) : V(vertices) {
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // ===== BFS =====

    // BFS returning distance and parent arrays
    pair<vector<int>, vector<int>> bfs(int src) {
        vector<int> dist(V, -1);
        vector<int> parent(V, -1);
        queue<int> q;

        dist[src] = 0;
        q.push(src);
```

```

        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    parent[v] = u;
                    q.push(v);
                }
            }
        }
        return {dist, parent};
    }

// Reconstruct path from src to dest using BFS parent
vector<int> getPath(int src, int dest) {
    auto [dist, parent] = bfs(src);
    if (dist[dest] == -1) return {}; // no path

    vector<int> path;
    for (int v = dest; v != -1; v = parent[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}

// BFS level order traversal
vector<vector<int>> bfsLevels(int src) {
    vector<vector<int>> levels;
    vector<bool> visited(V, false);
    queue<int> q;

    visited[src] = true;
    q.push(src);

    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        while (sz-- > 0) {
            int u = q.front(); q.pop();
            level.push_back(u);
            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        levels.push_back(level);
    }
    return levels;
}

// ===== DFS =====

```

```

// Recursive DFS
void dfsUtil(int u, vector<bool> &visited, vector<int> &order) {
    visited[u] = true;
    order.push_back(u);
    for (int v : adj[u])
        if (!visited[v])
            dfsUtil(v, visited, order);
}

vector<int> dfs(int src) {
    vector<bool> visited(V, false);
    vector<int> order;
    dfsUtil(src, visited, order);
    return order;
}

// Iterative DFS (avoids recursion depth issues)
vector<int> dfsIterative(int src) {
    vector<bool> visited(V, false);
    vector<int> order;
    stack<int> st;

    st.push(src);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (visited[u]) continue;
        visited[u] = true;
        order.push_back(u);
        // Push in reverse order to maintain same order as recursive
        for (int i = adj[u].size() - 1; i >= 0; i--)
            if (!visited[adj[u][i]])
                st.push(adj[u][i]);
    }
    return order;
}

// DFS for all vertices (handles disconnected graph)
vector<int> dfsAll() {
    vector<bool> visited(V, false);
    vector<int> order;
    for (int u = 0; u < V; u++)
        if (!visited[u])
            dfsUtil(u, visited, order);
    return order;
}

// ===== Cycle Detection =====

// Undirected graph cycle detection
bool hasCycleUndirected() {
    vector<bool> visited(V, false);
    for (int u = 0; u < V; u++) {
        if (!visited[u]) {

```

```

        // BFS with parent tracking
        queue<pair<int,int>> q;
        q.push({u, -1});
        visited[u] = true;
        while (!q.empty()) {
            auto [cur, par] = q.front(); q.pop();
            for (int v : adj[cur]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push({v, cur});
                } else if (v != par) {
                    return true; // cycle found
                }
            }
        }
    }
    return false;
}

// Directed graph cycle detection (DFS with recursion stack)
bool hasCycleDirectedUtil(int u, vector<bool> &visited, vector<bool>
&recStack) {
    visited[u] = true;
    recStack[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            if (hasCycleDirectedUtil(v, visited, recStack))
                return true;
        } else if (recStack[v]) {
            return true; // back edge = cycle
        }
    }
    recStack[u] = false;
    return false;
}

bool hasCycleDirected() {
    vector<bool> visited(V, false);
    vector<bool> recStack(V, false);
    for (int u = 0; u < V; u++)
        if (!visited[u])
            if (hasCycleDirectedUtil(u, visited, recStack))
                return true;
    return false;
}

};

int main() {
    GraphTraversal g(6);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);

```

```

g.addEdge(2, 4);
g.addEdge(3, 5);
g.addEdge(4, 5);

// BFS
auto [dist, parent] = g.bfs(0);
cout << "BFS distances from 0:\n";
for (int i = 0; i < 6; i++)
    cout << "0->" << i << " = " << dist[i] << "\n";

// Path reconstruction
auto path = g.getPath(0, 5);
cout << "Path 0->5: ";
for (int v : path) cout << v << " ";
cout << "\n";

// DFS
auto dfsOrder = g.dfs(0);
cout << "DFS from 0: ";
for (int v : dfsOrder) cout << v << " ";
cout << "\n";

// Cycle detection
cout << "Has cycle (undirected): " << g.hasCycleUndirected() << "\n"; // 1
(yes)
return 0;
}

```

## 9. Python Implementation

```

from collections import deque
from typing import List, Tuple

class GraphTraversal:
    def __init__(self, vertices: int):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def add_edge(self, u: int, v: int):
        self.adj[u].append(v)
        self.adj[v].append(u)

    # ===== BFS =====

    def bfs(self, src: int) -> Tuple[List[int], List[int]]:
        dist = [-1] * self.V
        parent = [-1] * self.V
        q = deque([src])
        dist[src] = 0

        while q:

```

```

        u = q.popleft()
        for v in self.adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)
        return dist, parent

def get_path(self, src: int, dest: int) -> List[int]:
    dist, parent = self.bfs(src)
    if dist[dest] == -1:
        return []
    path = []
    v = dest
    while v != -1:
        path.append(v)
        v = parent[v]
    return path[::-1]

def bfs_levels(self, src: int) -> List[List[int]]:
    levels = []
    visited = [False] * self.V
    q = deque([src])
    visited[src] = True

    while q:
        level = []
        for _ in range(len(q)):
            u = q.popleft()
            level.append(u)
            for v in self.adj[u]:
                if not visited[v]:
                    visited[v] = True
                    q.append(v)
        levels.append(level)
    return levels

# ===== DFS =====

def dfs_util(self, u: int, visited: List[bool], order: List[int]):
    visited[u] = True
    order.append(u)
    for v in self.adj[u]:
        if not visited[v]:
            self.dfs_util(v, visited, order)

def dfs(self, src: int) -> List[int]:
    visited = [False] * self.V
    order = []
    self.dfs_util(src, visited, order)
    return order

def dfs_iterative(self, src: int) -> List[int]:
    visited = [False] * self.V

```

```

order = []
stack = [src]
while stack:
    u = stack.pop()
    if visited[u]:
        continue
    visited[u] = True
    order.append(u)
    for v in reversed(self.adj[u]):
        if not visited[v]:
            stack.append(v)
return order

# ===== Cycle Detection =====

def has_cycle_undirected(self) -> bool:
    visited = [False] * self.V
    for u in range(self.V):
        if not visited[u]:
            q = deque([(u, -1)])
            visited[u] = True
            while q:
                cur, par = q.popleft()
                for v in self.adj[cur]:
                    if not visited[v]:
                        visited[v] = True
                        q.append((v, cur))
                    elif v != par:
                        return True
    return False

def has_cycle_directed(self) -> bool:
    visited = [False] * self.V
    rec_stack = [False] * self.V

    def dfs_cycle(u: int) -> bool:
        visited[u] = True
        rec_stack[u] = True
        for v in self.adj[u]:
            if not visited[v]:
                if dfs_cycle(v):
                    return True
            elif rec_stack[v]:
                return True
        rec_stack[u] = False
        return False

    for u in range(self.V):
        if not visited[u]:
            if dfs_cycle(u):
                return True
    return False

```

```
# Example
g = GraphTraversal(6)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(1, 4)
g.add_edge(2, 4)
g.add_edge(3, 5)
g.add_edge(4, 5)

dist, parent = g.bfs(0)
print("BFS distances from 0:", dist)

path = g.get_path(0, 5)
print("Path 0->5:", path)

print("DFS from 0:", g.dfs(0))
print("Has cycle:", g.has_cycle_undirected())
```

## 10. Code Explanation

### BFS:

- `dist` array initialized with -1 (unvisited).
- Queue starts with `src`, `dist[src] = 0`.
- For each node popped, iterate its neighbours. If unvisited, set distance = `dist[u] + 1`, set parent, push to queue.
- BFS ensures first visit = shortest path in unweighted graphs.
- `getPath()` reconstructs path by following parent pointers from dest back to src, then reversing.

### DFS:

- Recursive: `dfsUtil(u, visited, order)` marks visited, adds to order, recurses on unvisited neighbours.
- Iterative: uses explicit stack. Push nodes in reverse order to maintain similar order to recursive.
- `dfsAll()` handles disconnected graphs by starting DFS from each unvisited vertex.

### Cycle Detection:

- Undirected: During BFS/DFS, if we encounter a visited neighbour that is NOT the parent, there's a cycle.
- Directed: Uses `recStack[]` to track vertices currently on the recursion path. If we encounter a vertex already in `recStack`, it's a back edge → cycle.

## 11. Complexity Analysis

| Operation           | Complexity |
|---------------------|------------|
| BFS (single source) | $O(V + E)$ |
| DFS (single source) | $O(V + E)$ |

| Operation                          | Complexity |
|------------------------------------|------------|
| BFS Level Order                    | $O(V + E)$ |
| Path reconstruction                | $O(V)$     |
| Cycle detection (undirected)       | $O(V + E)$ |
| Cycle detection (directed)         | $O(V + E)$ |
| Space (visited, dist, queue/stack) | $O(V)$     |

## 12. Common Patterns

| Pattern                           | Identification                   | Approach                                                   | Problems                              |
|-----------------------------------|----------------------------------|------------------------------------------------------------|---------------------------------------|
| <b>Shortest Path (Unweighted)</b> | "Minimum edges", "minimum steps" | BFS                                                        | Shortest Path in Binary Matrix        |
| <b>Level Order Traversal</b>      | "Level by level", "zigzag"       | BFS with level tracking                                    | Binary Tree Level Order               |
| <b>Cycle Detection</b>            | "Detect cycle", "is it a tree?"  | DFS with parent (undirected), DFS with recStack (directed) | Course Schedule, Redundant Connection |
| <b>Topological Sort</b>           | "Ordering with dependencies"     | Kahn's algorithm (BFS with in-degree) or DFS post-order    | Course Schedule II                    |
| <b>Connected Components</b>       | "Count groups"                   | DFS/BFS from each unvisited vertex                         | Number of Provinces                   |
| <b>Bipartite Check</b>            | "2-coloring", "team division"    | BFS/DFS with alternating colors                            | Is Graph Bipartite?                   |
| <b>Flood Fill</b>                 | "Fill connected region"          | DFS/BFS from a cell                                        | Flood Fill, Number of Islands         |

## 13. Common Mistakes

- **BFS:**
  - Marking visited when popping instead of pushing — causes multiple pushes of same node.
  - Using recursion for BFS — BFS is inherently iterative.
  - Not handling disconnected components (BFS from one source misses other components).
- **DFS:**
  - Recursion stack overflow for large  $V$  ( $V > 10^5$ ) — use iterative DFS.
  - Not using `recStack[]` for cycle detection in directed graphs (visited alone is not enough).
  - Forgetting to backtrack in `recStack[]` after DFS returns from a node.
  - In undirected cycle detection, not checking parent — every back edge seems like a cycle.
- **General:**

- Not resetting visited array between test cases.
- Using `int` for dist when `INF = INT_MAX` and adding 1 causes overflow.
- Not handling self-loops (a node connected to itself) in cycle detection.

## 14. Edge Cases

- **Single vertex, no edges** — BFS/DFS just visits that vertex.
- **Disconnected graph** — BFS/DFS from one source only reaches one component.
- **Complete graph** — all vertices at distance 1 from source.
- **Graph with self-loops** — self-loop is a cycle in directed graphs.
- **Graph with parallel edges** — no issue for traversal (visited check handles it).
- **Tree** — no cycles,  $V-1$  edges, connected.
- **Empty graph** ( $V = 0$ ) — nothing to traverse.

## 15. Variations

| Variation                | What Changes                                        | When Used                              | Importance   |
|--------------------------|-----------------------------------------------------|----------------------------------------|--------------|
| <b>Iterative DFS</b>     | Uses explicit stack instead of recursion            | Avoids stack overflow for deep graphs  | High         |
| <b>Multi-source BFS</b>  | Multiple initial sources in queue                   | Nearest distance from a set of sources | Very High    |
| <b>Bidirectional BFS</b> | BFS from both source and destination simultaneously | Faster path finding in large graphs    | Medium       |
| <b>0-1 BFS</b>           | Deque-based BFS for 0/1 weighted edges              | Shortest path when weights are 0 or 1  | High         |
| <b>A* Search</b>         | BFS with heuristic function                         | Pathfinding with domain knowledge      | Low (not CP) |

## 16. Related Concepts

- **Dijkstra's Algorithm** — Weighted graph version of BFS for shortest path.
- **Topological Sort** — DFS post-order or Kahn's algorithm (BFS on in-degree).
- **Strongly Connected Components** — Kosaraju's uses two DFS traversals.
- **Tree Traversals** — Preorder, inorder, postorder are special cases of DFS on trees.
- **Dynamic Programming** — BFS/DFS on DAG is equivalent to DP.

## 17. Practice Problems

### Easy

- **Flood Fill** — LeetCode (733) — DFS/BFS on grid
- **Maximum Depth of Binary Tree** — LeetCode (104) — BFS/DFS on tree

### Medium

- **Number of Islands** — LeetCode (200) — DFS/BFS on grid components

- **Course Schedule** — LeetCode (207) — Cycle detection (directed)
- **Word Ladder** — LeetCode (127) — BFS on implicit graph

## Hard

- **Shortest Path Visiting All Nodes** — LeetCode (847) — BFS + bitmask DP
- **Minimum Obstacle Removal** — LeetCode (2290) — 0-1 BFS
- **Alien Dictionary** — LeetCode (269) — Topological sort via DFS

## 18. Interview Explanation

"Graph traversal algorithms are BFS and DFS. BFS uses a queue to explore nodes level by level, giving shortest paths in unweighted graphs in  $O(V+E)$  time. I use it for shortest path, level order, and multi-source problems. DFS uses recursion or a stack to explore deep first. I use it for cycle detection, topological sort, and connected components. For cycle detection in undirected graphs, I track the parent to avoid false positives. For directed graphs, I also maintain a recursion stack to detect back edges. The key difference is that BFS is optimal for shortest path, while DFS is better for exploring all paths."

## 19. Revision Notes

- BFS: queue, level by level, shortest path.
- DFS: stack/recursion, deep first, backtracking.
- BFS time:  $O(V+E)$ , DFS time:  $O(V+E)$ .
- BFS space:  $O(V)$  (queue can be large).
- DFS space:  $O(V)$  (recursion stack or explicit stack).
- Cycle detection:
  - Undirected: **visited** + **parent** check.
  - Directed: **visited** + **recStack** (back edge).
- Mark visited when pushing to queue (BFS), when popping from stack (iterative DFS), or when entering (recursive DFS).
- Multi-source BFS: push all sources initially.
- 0-1 BFS: deque, push front for weight 0, push back for weight 1.

## 20. Final Cheat Sheet

| Aspect                     | BFS             | DFS               |
|----------------------------|-----------------|-------------------|
| Data structure             | Queue           | Stack / Recursion |
| Traversal order            | Level by level  | Deep first        |
| Shortest path (unweighted) | ✓ Yes           | ✗ No              |
| Cycle detection            | ⚠ (with parent) | ✓ (natural)       |
| Topological sort           | ✓ (Kahn's)      | ✓ (post-order)    |
| Connected components       | ✓               | ✓                 |
| Time complexity            | $O(V+E)$        | $O(V+E)$          |

| Aspect           | BFS                        | DFS                               |
|------------------|----------------------------|-----------------------------------|
| Space complexity | $O(V)$                     | $O(V)$                            |
| Key trap         | Mark visited on push       | Stack overflow (recursion)        |
| Must know        | Shortest path, level order | Cycle detection, topological sort |

**End of Graphs Guide** — Master these concepts, practice the problems, and you'll be well-prepared for any graph question in placements and competitive programming.